



Master's Degree in Mechatronics Engineering

FINAL DISSERTATION

Machine Learning and Control Theory for Accurate and Reliable Temperature Estimation in Power Transformers

ALEX PEGORARO
257770

UniTrento Supervisor:
LUCA ZACCARIAN

KTH Supervisor:
FEDERICA BRAGONE

Industrial Supervisor:
**CARL-JOHAN TOR
LANERYD**

KTH Examiner:
**MATTHIEU RAPHAEL
BARREAU**

Academic Year:
2025/2026

Machine Learning and Control Theory for Accurate and Reliable Temperature Estimation in Power Transformers

ALEX PEGORARO

Master's Programme, Mechatronics Engineering, 120 credits

Date: September 8, 2026

Supervisors: Federica Bragone, Luca Zaccarian, Carl-Johan Tor Laneryd

Examiner: Matthieu Raphael Barreau

Examiner School: School of Electrical Engineering and Computer Science

Host company: Hitachi Energy Ltd

Swedish title: Maskininlärning och Reglerteknik för Noggrann och Tillförlitlig
Temperaturuppskattning i Krafttransformatorer

Abstract

Power Transformers are critical elements of the electrical network; the elevate temperatures they reach while processing current can provoke an excessive degradation of their most delicate components, resulting in premature system aging. Hence, it is of crucial importance to properly model the evolution of the *hotspot temperature* reached by the machine, in order to estimate the associated *loss of life* and optimize the maintenance schedule accordingly. The project begins with the implementation of the “IEC Model”, a popular mathematical representation of Power Transformer’s thermal behavior, and tests the efficacy of possible improvement techniques that do not compromise the simplicity of the model.

From the field of *Control Theory* the Luenberger Observer and Kalman Filter are tested. They have an extremely similar effect on the system, and cause an error reduction of -65% over the Mean Absolute Error (MAE) and of -53% over the Root Mean Square Error (RMSE). On the other hand, the Recurrent Neural Network is selected to represent the field of *Machine Learning*. Six architecture are tested, and the most effective one gains a reduction of -48% over the MAE and -45% over the RMSE. Finally, the two domains are fused into an hybrid approach which obtains the best performance, -75% over MAE and -66% over RMSE, demonstrating that different techniques can be stacked without one shadowing the effects of the other.

The research leaves space for many types of continuations, including the employment of simulation algorithms guaranteeing better numerical precision, the implementation on physical Power Transformer machines, and the experimentation of additional Recurrent Neural Network Architectures.

Keywords

Power transformers, State estimation, Machine learning, Recurrent neural networks, Control theory, State observers, Kalman filters

Sammanfattning

Krafttransformatorer är kritiska komponenter i elnätet; de förhöjda temperaturer de uppnår när de leder ström kan orsaka en överdriven nedbrytning av transformatorns känsligaste komponenter, vilket leder till förtida åldring av systemet. Därför är det avgörande att korrekt modellera utvecklingen av den *hotspot-temperatur* som transformatorn uppnår, för att kunna uppskatta den tillhörande *livslängdsförbrukningen* och därigenom optimera underhållsplaneringen. Detta projekt inleddes med implementeringen av "IEC-modellen", en vanligt förekommande matematisk representation av krafttransformatorers termiska beteende, och prövade effektiviteten hos möjliga förbättringstekniker som inte äventyrar modellens enkelhet.

Från området *reglerteknik* testades Luenbergerobservatören och Kalmanfiltret. Dessa hade en mycket likartad effekt på systemet och gav en felminskning på -65% för medelabsolutfelet (eng. MAE) och på -53% för rotmedelkvadratfelet (eng. RMSE). Å andra sidan har det rekurrenta neurala nätverket valts för att representera området *maskininlärning*. Sex arkitekturer har testats, och den mest effektiva uppnådde en minskning på -48% för MAE och -45% för RMSE. Slutligen kombinerades de två domänerna i en hybridmetod som uppnådde den bästa prestandan, -75% för MAE och -66% för RMSE, vilket visar att olika tekniker kan läggas ovanpå varandra utan att den ena överskuggar den andras effekter.

Forskningen lämnar utrymme för många typer av fortsättningar, däribland användning av simuleringsalgoritmer som garanterar bättre numerisk precision, implementering på verkliga krafttransformatorer samt experimenterande med ytterligare arkitekturer för rekurrenta neurala nätverk.

Nyckelord

Krafttransformatorer, Tillståndsskattning, Maskininlärning, Rekurrenta neurala nätverk, Reglerteknik, Tillståndsobservatörer, Kalmanfilter

Sommarlo

I Trasformatori di Potenza sono elementi critici di una rete elettrica; le elevate temperature a cui sono sottoposti mentre processano corrente possono provocare un'eccessiva degradazione dei loro componenti più delicati, risultando in un prematuro invecchiamento del sistema. Per questo motivo, è di cruciale importanza modellare correttamente l'evoluzione della *temperatura massima* della macchina, in modo da poter stimare la *perdita di vita* associata ed ottimizzare in conseguenza le operazioni di manutenzione. Questo progetto parte dall'implementazione del "Modello IEC", una popolare rappresentazione matematica del comportamento termico dei Trasformatori, per poi testare l'efficacia di possibili tecniche di miglioramento che non compromettano la semplicità del modello.

Dall'abito della *Teoria dei Controlli* sono stati testati l'Osservatore di Luenberger e il Filtro di Kalman. Hanno ottenuto effetti estremamente simili sul sistema, causando una riduzione dell'errore del -65% riguardo l'Errore Assoluto (eng. MAE) e del -53% riguardo la radice dello Scarto Quadratico Medio (eng. RMSE). Dall'altro lato, le Reti Neurali Ricorrenti sono state scelte come rappresentanti dell'ambito dell'*Apprendimento Automatico*. Sei architetture sono state testate, e la più efficace ha ottenuto una riduzione del -48% sul MAE e -45% sul RMSE. I due domini sono inoltre stati fusi in un approccio ibrido che ha ottenuto le migliori prestazioni, -75% di MAE e -66% di RMSE, dimostrando che tecniche differenti possono essere usate insieme senza che una sovrasti gli effetti dell'altra.

La ricerca lascia spazio a numerosi proseguimenti, includendo l'utilizzo di algoritmi di simulazione con migliore precisione numerica, l'implementazione su dispositivi fisici per la Trasformazione di Potenza, e la sperimentazione di nuove architetture per le Reti Neurali Ricorrenti.

Parole Chiave

Trasformatori di potenza, Stima dello stato, Apprendimento automatico, Reti neurali ricorrenti, Teoria dei controlli, Osservatori dello stato, Filtri di Kalman

Institutional Acknowledgments

I would like to thank the entire team that assisted me during the development of the project. I thank Matthieu Barreau and Federica Bragone for the weekly meetings that shaped the evolution of my thesis. I thank Carl-Johan Tor Laneryd for sharing his expertise on Power Transformers and giving me the opportunity to present the research in a company meeting. I thank Luca Zaccarian for having introduced me to the group and for his role as a linkage between the two universities. I also thank Tim Gradnik for the support and clarifications concerning the CIGRE dataset.

Stockholm, September 2026

Alex Pegoraro

Personal Acknowledgments

Voglio ringraziare tutte le persone che mi sono state vicino e mi hanno supportato in questi cinque anni di studi, a partire dalla mia famiglia. Ringrazio Mamma Nives e Papà Agostino per aver sempre creduto in me, e ringrazio Ransford e i miei cugini Alessandro e Luca per essere sempre stati presenti.

Ringrazio poi gli amici con cui ho condiviso sia gli studi che lo svago. Un grazie speciale a Alan, Bosco, Sofia e Paolo, ma anche a tutti gli altri compagni di feste e di progetti, tra cui ricordo Silvia, Filippo, Giulia, Fabio, Lorenzo Pattaro Zonta, Marogna, Mosè, Samuel e anche Luca del FabLab.

Ringrazio la *Sala d'Arme Achille Marozzo di Trento* e tutte le persone che vi ho conosciuto. Ringrazio l'istruttore Andrea e i tecnici Rick, Alex, Davide e Lorenzo (il migliore tassista di Trento), oltre che tutti gli altri schermitori con cui ho duellato, di cui voglio menzionare in particolare Giovanni, Giorgio, Paolo, Leonardo, Letizia, Selene e Gugliemo.

I would like to thank *The English Improv Theater of Stockholm* for having welcomed me and permitted me to be your music leader. Some of you were with me in the Spex, like Anton (and all his ties), Andrés, Adam, Kaisa, Marco, Ashmit and Max. Others played role games with me, I mean Nicolas, Corentin, Saga and Anveil. Many others like André and Rune made the Improv sessions special with their presence.

Ringrazio tutti i professori che ho avuto, a partire dal Liceo, passando dalla Triennale fino ad arrivare alla Magistrale. Voglio menzionare in particolare tre professori universitari che si sono distinti per la passione e l'impegno che mettono nella loro materia: il prof. Roberto Passerone, che forse non darà importanza alle pause ma di sicuro dà importanza ai suoi studenti, il prof. Alberto Montresor, l'unico capace di riempire la A101 usando solo il suo carisma, e il prof. Luca Zaccarian, che dopo ogni lezione finiva puntualmente per fare una seconda lezione da quanti approfondimenti gli chiedevo.

Infine, voglio ringraziare di cuore due persone senza cui questa tesi non sarebbe davvero stata possibile. Voglio ringraziare Federica Lorenzini per avermi supportato quando ero lontano da casa fra i ghiacci della Scandinavia, e voglio ringraziare Andrea Camaioni per avermi fatto appassionare al mondo dell'informatica ed avermi ispirato a scegliere questo percorso.

Grazie mille a tutti quanti.

Stockholm, September 2026

Alex Pegoraro

Contents

- 1 Introduction 1**
 - 1.1 Background 1
 - 1.2 Problem Definition 3
 - 1.3 Purpose 4
 - 1.4 Goals 4
 - 1.5 Research Methodology 5
 - 1.6 Delimitations 7
 - 1.7 Structure of the Thesis 7
- 2 Background 9**
 - 2.1 Power Transformer Systems 9
 - 2.2 Deterministic Control Theory 15
 - 2.3 Recurrent Neural Networks 18
 - 2.4 Related Works 20
- 3 Research Methodology 23**
 - 3.1 Research Paradigm 23
 - 3.2 Research Process 24
 - 3.3 The Dataset 25
 - 3.4 Testing Environment 25
 - 3.5 Quality Assurance 26
 - 3.6 Evaluation Framework 27
 - 3.7 Comparison Metrics 28
 - 3.8 System Documentation 30
- 4 System Design 31**
 - 4.1 From IEC to SSR 31
 - 4.2 Luenberger Gain 33
 - 4.3 RNN Architecture 33
 - 4.4 Data Preprocessing 37

4.5	Software Infrastructure	38
5	Experiments and Results	41
5.1	Deterministic Improvements	41
5.2	RNN Training	42
5.3	Generalization Capabilities of RNNs	45
5.4	A Hybrid Approach	47
5.5	Loss of Life	48
6	Discussions	51
6.1	Luenberger vs Kalman	51
6.2	A Two-stages Approach to RNNs	52
6.3	The Problem of Overfitting	52
6.4	Feasibility of the Hybrid Approach	53
6.5	Improvements on the Loss of Life	53
7	Conclusions and Future Works	55
7.1	Conclusions	55
7.2	Research Answer	56
7.3	Limitations	56
7.4	Future Works	56
7.5	Reflections	58
	References	61
A	Observability in RNNs	65
A.1	Definition of Observability	65
A.2	Observability Improvements with RNNs	66

List of Figures

1.1	Schematic of a Power Transformer as considered in the thesis .	2
2.1	System Diagram of a Power Transformer as described by the IEC Model in the Differential Formulation	14
2.2	Scheme of an Artificial Neural Network	19
2.3	Scheme of a Recurrent Neural Network (RNN)	20
4.1	Full-time training	36
4.2	Batch training	36
4.3	The dataset used for the experiments	37
5.1	Errors on the test dataset for the Deterministic Control Theory (DCT) improvements, with Kalman Filter's plot identical to Luenberger	42
5.2	Errors on the test dataset for two representative RNNs	46
5.3	Simulations on the test dataset with hybrid improvements	48
5.4	Loss of Life on the test dataset for the original and hybrid model	49

List of Tables

2.1	Parameters of the IEC Model	11
3.1	Hardware specifications of the computer used to run the simulations	25
4.1	List of RNN Architectures tested in the project	39
5.1	Detailed error metrics on the test dataset for the DCT improvements	42
5.2	Error reductions on the train dataset for the RNN architectures	43
5.3	Error metrics and error reductions on the test dataset for the RNN architectures	46
5.4	Error metrics and error reductions on the test dataset for the hybrid improvements	47
5.5	Loss of life metrics on the test dataset for the hybrid improvements	49

List of Acronyms and Abbreviations

DCT	Deterministic Control Theory
GNU- GPL	GNU's General Public License
MAE	Mean Absolute Error
ML	Machine Learning
MSE	Mean Square Error
ODE	Ordinary Differential Equation
RMSE	Root Mean Square Error
RNN	Recurrent Neural Network
SDG	Sustainable Development Goal
SSR	State Space Representation
UN	United Nations

Chapter 1

Introduction

Power Transformer are crucial components of an electrical network. Unfortunately, their maintenance can be difficult, and faults like the loss of insulation in the oil-impregnated paper often require an entire system replacement, which causes high costs to the company as well as long productions downtimes [1, 2]. It is then important to properly model the system behavior in order to predict and control the value of the *hotspot temperature*, i.e. the highest temperature reached by the oil. The proper collection of the data is essential to maximize the lifespan of the most delicate components of the machine. The thesis explores possible improvements to the *IEC Model* [3], commonly used for temperature estimation, seeking them in **Deterministic Control Theory (DCT)** as well as the **Machine Learning (ML)** field, expanding the work in [4] and [1].

1.1 Background

Power Transformers

The system examined by the project is a Power Transformer [5]. Many models have been developed to capture their behavior under the thermal perspective, one of them being the *IEC Model* [3], quite diffused in the sector for its accuracy-simplicity trade off. Figure 1.1 presents the aspects of interest when dealing with temperature estimation. Power Transformers can be considered *dynamical systems* that concern the evolution of many variables, two of which are the focus of the thesis: the Top-oil temperature and the Hotspot temperature. Their values are obtained by a feedback on their previous state, together with two direct inputs: the ambient temperature and the Load

Factor, a pure number correlated to the amount of current processed by the system. Usually the dynamic of the variables is mathematically formulated in continuous-time, but the simulation algorithms often require a conversion to the discrete-time domain. It is the reason why Figure 1.1 includes a Sample & Hold block within the feedback, which indicates we can read values from the system at discrete-time intervals.

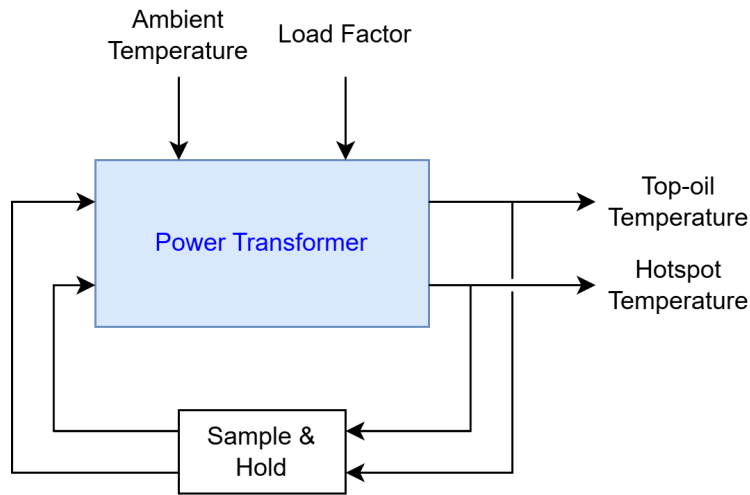


Figure 1.1: Schematic of a Power Transformer as considered in the thesis

Control Theory

The system is first be converted into the **State Space Representation (SSR)** [6] typical of Control Theory, described by the following set of equations:

$$\begin{cases} x'_t = Ax_t + Bu_t \\ y_t = Cx_t \end{cases} \quad (1.1)$$

Where u_t is the vector of inputs, y_t the vector of outputs, and x_t the vector of system states. The matrices A , B and C encodes the system dynamic and are derived by the coefficients of the **Ordinary Differential Equations (ODEs)** describing the model, which is the IEC Model. Thanks to a standard representation, it is possible to seamlessly employ the following Control Theory techniques:

- *Euler Discretization* [7]: It is one of the possible methods employed to convert the **SSR** from the continuous domain to the discrete one. It does not improve the accuracy of the system, but is often required to run the simulation.
- *Luenberger Observer* [8]: A technique that employs an adjustable gain to provide a better estimation of the system state in case some of the outputs are observable.
- *Kalman Filter* [9]: In case the parameters of the noise affecting the measurements are known, the algorithm provides a state estimation more accurate than the observer, at the price of a longer computational time.

Machine Learning

The **ML** paradigm selected for the project is the **Recurrent Neural Network (RNN)** [10]. It is a type of Neural Network particularly suited for time series analysis, the problem faced by the research. Just like the Control Theory approach, **RNNs** maintain an hidden *state vector*, updated over a discrete number of steps. The update step is regulated by a set of parameters, repeatedly tuned by the network to minimize the difference between the predicted series and the given one.

1.2 Problem Definition

The project aims to put itself as the first step towards the generation of a new dynamical model for Power Transformer, particularly suited for temperature variations. In order to do so, the IEC Model is used as baseline, and a couple of techniques from both **ML** and **DCT** are employed to increase its performances.

Research Question

The research question that guided the development of the work is the following:

Which Automatic Control techniques, selected from Deterministic Control Theory as well as Machine Learning, can be used to extend the 'IEC Model', improving its accuracy in the hotspot temperature estimation of Power Transformer on historical datasets with time-varying load factor?

1.3 Purpose

Hitachi Energy Ltd owns a Power Transformer facility in Västerås, and uses two models to control the devices, namely “Annex G” and “IEC”. Both the models provide a suitable description of the Power Transformers’ temperatures in steady state conditions, but struggle to capture their transient behavior. They are suitable when the load factor is uniform, but prone to errors in case the inputs present high variance.

Unfortunately, renewable energy resources, like wind or sun, often presents intermittent loads rather than uniform, so Power Transformer connected to them are prone to be mis-modeled and risk higher degradation. Hence, Hitachi Energy Ltd is looking for more accurate Power Transformer models, whether they are analytical or **ML** driven. They can them to predict the hotspot temperature in the situations where the simple models fail, and empower actions to keep it within the desired limits, increasing the living time of their machines. The thesis is a notable step in such direction.

Ethics, Sustainability and Social Issues

The project does not rise any ethical issue since no human subject or personal data is involved in the analysis. Similarly, social issues are barely touched by the research. There is a possibility that the location and the amount of current processed by a Power Transformer may create spatial inequalities among the users regarding the amount of energy they receive, but the thesis addresses neither of the two characteristics, because it concerns only the modeling of the implant rather than its management.

On the other hand, sustainability is an aspect positively addressed by the project: improving the estimation of the thermal behavior of an implant can lead to a better maintenance schedule for the Power Transformer machines, increasing the overall sustainability of the facilities. The project contributes to the **United Nations (UN) Sustainable Development Goals (SDGs)** [1] number 7 “Affordable and Clean Energy” and number 9 “Industry, Innovation and Infrastructure”.

1.4 Goals

The goal of the project is to modify the IEC Model traditionally employed by Hitachi Energy Ltd through the usage of a mixture of **DCT** and **ML** techniques to improve its estimation performance on historical datasets. Instead of

implemented a single approach, a variety of different modifications have been tested in order to determine which ones are the most efficient and which ones appears to be negligible. From the perspective of the company, they want to receive a list of the most reasonable techniques to improve the IEC Model, helping them directing their research and future projects towards the most promising direction.

Objectives

To be able to answer the Research Question, the project has been decomposed into the following objectives:

- *Documentation*: Document the state of the art in the field of Power Transformer and their simulation models. The objective is addressed in Chapter 2.
- *Model Improvement*: A set of model improvement techniques has been selected and implemented, from both DCT and ML. The objective is described in Chapter 4.
- *Model Evaluation*: The impacts of the modifications have been evaluated and compared to the baseline model through the defined performance metrics. The objective is addressed in Chapters 5 and 6.

1.5 Research Methodology

The research adopted a quantitative experimental research method [12], which means the conclusions are drawn from the analysis of the numerical results of a group of experiments. In such framework, the research approach was centered on the evaluation of the eventual improvements a selected set of control algorithms can give to the IEC Model, through a series of experiments run on historical Power Transformer datasets. The research process started with a strong focus on the literature review about the systems, in order to properly understand such model. Then, it moved towards the implementation of each considered technique and the evaluation of its performance on the data. Finally, the conclusions are stipulated by comparing the performance with the one obtained by the unmodified IEC Model on the same scenario.

The Method is particularly suited for the project for the great flexibility it provides regarding the algorithms and the performance metrics to select. Thanks to it, other researchers can apply the same exact workflow to novel

control techniques and/or with other performance indices; their results would be immediately comparable to the ones discussed in the thesis, facilitating the future developments of the research.

List of Resources

The most important resources required by the project are the datasets containing top-oil temperatures' and hotspot temperatures' time series, which have been provided by Hitachi Energy Ltd. The software used to process them is Python, and in particular the NumPy, pyTorch, Pandas and Matplotlib libraries. The other resource available to the student is the specialized personal of Hitachi Energy Ltd, that have provided consultancy on the system specifications.

The code has been run locally on a *Lenovo Legion C7* Personal Computer, hence no Cloud Computing platform has been required. No other physical equipment has been required. Finally, due to the experimental nature of the project, no interview to human subjects has been required to be carried out.

Evaluation

The degree project's fulfillment condition is the implementation of the following algorithms and their comparison to the baseline IEC Model:

- Luenberger Observer [8]
- Kalman Filter [9]
- The RNN [10] Architectures presented in Chapter 4

The quantitative measures employed in the comparisons are the **Mean Absolute Error (MAE)** and the **Root Mean Square Error (RMSE)** [13]. Both of them are time series comparison methods that outputs a positive scalar value which encodes the difference between the series of ground truth samples and the corresponding series of simulation results .

Scientific Relevance

The work is scientifically relevant due to the comparison it provides between **DCT** and **ML** models in the system simulation field. From the academic perspective, the main contributions of the project are to outline the intrinsic consequences given by the selection of the two approaches, highlight the

similarities and differences among them, and explore possible way of using them together. Its contribute is given by a direct evaluation of the impact the two domains cause on a practical example, the Power Transformer.

Innovation

The project provides innovation to the researchers and the companies that deal with Power Transformer simulations. The contents of the thesis are useful to them in the situation when the traditional models like the IEC one fail to represent a dataset, since they present improvement techniques that may enhance the precision of their simulations. In particular, the strength of the thesis is that it directly compares **ML** and **DCT**, helping them deciding which of the two approaches is the most suitable for their situation.

1.6 Delimitations

The main limitation of the project is given by the initial selection of models to be examined by the student: algorithms non included in the pool are not examined, as well as baseline models different from IEC. Hence, the project determines the best improvement that IEC Model can receive *among the ones described in Chapter 4*, not the best Power Transformer model in an absolute sense. Another limitation is that the algorithms are be evaluated by *simulations on historical datasets* only. Therefore, there are be no physical tests on the real Power Transformer plant, it is left as a Future Work.

1.7 Structure of the Thesis

Chapter 1 provides general introduction to all the topics covered by the research. The rest of the thesis is structured as follow. Chapter 2 provides the necessary background knowledge to understand the rest of the document, providing the relevant literature study about Power Transformer systems, the selected control Theory algorithms and some general aspects about **RNNs**. Chapter 3 outlines the Scientific Methodology that guided the evolution of the project.

Chapter 4 explains in details the structure of the code used for the simulation, with particular attention given to the composition of the selected **RNN** architectures. Chapter 5 is where the experimental results are provided through plots and tables. Chapter 6 is dedicated to the interpretation of such

results and their consequences on the system. Chapter 7 presents possible future works that can be built on top of the thesis, together with the general conclusions and reflections on the project. Finally, all the References are provided.

Chapter 2

Background

The chapter provides the background about all the fundamental building blocks necessary to construct the rest of the project. Section 2.1 presents the equations governing a Power Transformer System according to the IEC Model, but not how it has been reinterpreted according to the Control Theory framework, since it is addressed in Section 4.1. Section 2.2 describes the deterministic algorithms exploited in the research; Section 2.3 explores the principles of a RNN, and finally Section 2.4 gives an overview of related works.

2.1 Power Transformer Systems

Power Transformer are components of an Electrical Power Grid [4]. While performing their activity of processing electrical current, they have the side effect of producing heat [5]. The system is designed to operate in a certain temperature range, and when it becomes too hot some components may suffer additional degradation, which can lead to a premature system aging [4]. Additionally, maintenance operations to elements like the oil-impregnated paper are extremely expensive in both money and system downtimes, since they require the entire machine disassembly.

Consequently, being able to foresee temperature peaks in order to take preventive actions before irreversible damages are carried has become a routine practice in all Power Transformer facilities [14], with many models arisen to predict the thermal behavior of such machines. Hitachi Energy Ltd has historically adopted two mathematical models for their implant in Västerås, called “IEC” and “Annex G”. In the project, the company requested an analysis of the former.

IEC Model

The IEC Model is a mathematical model for the estimation of Power Transformers' thermal behavior, developed by the "International Electrotechnical Commission" and widely adopted by the companies around the world. IEC released many implementation variants, the one followed in the thesis is "IEC 60076-7:2005" [3]. It is very popular in the sector, to the point that it has been included in national standards like "The Indian Standard on Power Transformers" [15]. The information presented in the rest of the section is based on the International Standard [3] where not specified otherwise.

Model Variables

The IEC Model studies the evolution of two different kind of temperatures in a Power Transformer:

Top-oil Temperature $\theta_o(t)$ [°C] : The temperature the liquid assumes at the top of the oil tank.

Hotspot Temperature $\theta_h(t)$ [°C] : The hottest temperature assumed by the liquid; it can be anywhere along the height of the oil tank.

The most important for maintenance purposes is the Hotspot, since it is the highest temperature that determines the most relevant damage to the machine. Another variable is strongly correlated with θ_h :

Loss of Life $L(\theta_h)$ [min] : Is a measure of the wearing level of a machine component. When it reaches a certain threshold, dependent on the system, a maintenance action is triggered.

To compute the values, the model requires two kinds of inputs:

Load Factor $K(t)$ [p.u.] : A variable that encodes the amount of current traversing the transformer. Physically, it represents the ratio between two currents, hence it is a pure unit.

Ambient Temperature $\theta_a(t)$ [°C] : The temperature of the air mass surrounding the Power Transformer.

Additionally, there are some parameters that differentiate an individual machine from another one; some of them are design choices selected during manufacturing, others are measured in the testing phase of the plant. The IEC

Symbol	Unit	Parameter Name
$\Delta\theta_{or}$	[°C]	Top oil temperature rise in steady state at rated losses
$\Delta\theta_{hr}$	[°C]	Hotspot to top oil gradient at rated current
τ_o	[min]	Average oil time constant
τ_w	[min]	Winding time constant
R	[]	Loss ratio
x	[]	Exponential power of total losses
y	[]	Winding exponent
k_{11}	[]	Fitting constant 1-1
k_{21}	[]	Fitting constant 2-1
k_{22}	[]	Fitting constant 2-2

Table 2.1: Parameters of the IEC Model

Model considers them constant and known at the beginning of the simulation. Table 2.1 reports the list of such parameters.

Finally, it is a common practice to define two functions of the Load Factor: K_x and K_y . Their introduction captures the nonlinearities of the Model, so that what remains expressed in their terms is a Linear system. Their definition is the following:

$$K_x(t) = \left(\frac{1 + K(t)^2 R}{1 + R} \right)^x, \quad K_y(t) = K(t)^y \quad (2.1)$$

Exponential Formulation

The Exponential Formulation of the IEC Model is used when the ambient temperature is considered constant, and the Load Factor changes according to a step:

$$\theta_a(t) = \theta_a \quad \forall t, \quad K(t) = \begin{cases} K^{\text{prev}} & t < 0 \\ K & t \geq 0 \end{cases} \quad (2.2)$$

The case is quite common in the normal operation of a Power Transformer, and allows to model the output temperatures with two explicit functions (sometimes combined into a single one, like in [14]):

$$\begin{cases} \theta_o(t) = \theta_a + \Delta\theta_{or}K_x + (\Delta\theta_{oi} - \Delta\theta_{or}K_x)f_o(t) \\ \theta_h(t) = \theta_o(t) + \Delta\theta_{hr}K_y + (\Delta\theta_{hi} - \Delta\theta_{hr}K_y)f_h(t) \end{cases} \quad (2.3)$$

Where $\Delta\theta_{oi}$ and $\Delta\theta_{hi}$ are initial condition parameters, while f_o and f_h are two exponential functions defined as follows:

$$\begin{cases} f_o(t) = \exp\left(\frac{-t}{k_{11}\tau_o}\right) \\ f_h(t) = k_{21} \exp\left(\frac{-t}{k_{22}\tau_w}\right) + (1 - k_{21}) \exp\left(-t\frac{k_{22}}{\tau_o}\right) \end{cases} \quad (2.4)$$

We can observe both of the functions have value 1 when the step occurs ($t = 0$) and converge to 0 when the system is at regime, since they represent the transient behavior of the outputs. We can see f_o is a plain exponential, while f_h is the weighted sum of two exponentials, which means θ_h presents second order effects. Hence, we can give a concrete interpretation to what has been expressed in formulae so far:

When K has an instantaneous change, the difference between θ_o and θ_a is driven from $\Delta\theta_{oi}$ to $\Delta\theta_{or}K_x$ with a I order transient, while the difference between θ_h and θ_o moves from $\Delta\theta_{hi}$ to $\Delta\theta_{hr}K_y$ with a II order transient.

One last thing needs to be defined to be able to use such formulation: the values of the initial conditions $\Delta\theta_{oi}$ and $\Delta\theta_{hi}$. Usually, the *Steady State Assumption* is employed: Since the time between two changes in the load is often much less than the time required for the transient expiration, we can assume that when a step happens the system is at regime with respect to the previous value of the Load. It means that:

$$\begin{cases} \Delta\theta_{oi} = \Delta\theta_{or}K_x^{\text{prev}} \\ \Delta\theta_{hi} = \Delta\theta_{hr}K_y^{\text{prev}} \end{cases} \quad (2.5)$$

Differential Formulation

The Exponential Formulation ceases to be valid in the general case where $\theta_a(t)$ and $K(t)$ are not constants. It is the reason why [3] introduces another formulation for the IEC Model, where the exponential functions are substituted by the **ODEs** generating them. It no more uses an explicit function dictating the value of the outputs, but it employs a formula that describes their local changes (i.e. their derivative) respect to their current value.

For θ_o the conversion is immediate, since it is a plain exponential which can be directly mapped to a I order **ODE**. On the contrary, θ_h would require

a II order **ODE**, with the drawback of having to manage its second derivative as well. To avoid the problem, we can split the hotspot into two independent variables θ_{h1} and θ_{h2} :

$$\theta_h(t) = \theta_o(t) + \theta_{h1}(t) + \theta_{h2}(t) \quad (2.6)$$

If the variables are selected so that they contain only one exponential each, they can both be a solution of a I order differential equation. Hence, such variable change allows us to take care of only I order **ODEs**, and give an easy way to reconstruct the hotspot with a sum when needed.

The formulation we obtain, is the following:

$$\begin{cases} k_{11}\tau_o \frac{d}{dt}\theta_o(t) = -\theta_o(t) + \theta_a(t) + \Delta\theta_{or}K_x(t) \\ \theta_o(0) = \theta_a(0) + \Delta\theta_{or}K_x(0) \end{cases}$$

$$\begin{cases} k_{22}\tau_w \frac{d}{dt}\theta_{h1}(t) = -\theta_{h1}(t) + k_{21}\Delta\theta_{hr}K_y(t) \\ \theta_{h1}(0) = k_{21}\Delta\theta_{hr}K_y(0) \end{cases} \quad (2.7)$$

$$\begin{cases} \frac{\tau_o}{k_{22}} \frac{d}{dt}\theta_{h2}(t) = -\theta_{h2}(t) + (1 - k_{21})\Delta\theta_{hr}K_y(t) \\ \theta_{h2}(0) = (1 - k_{21})\Delta\theta_{hr}K_y(0) \end{cases}$$

The three initial conditions have been obtained using a consequence of the Steady State Assumption: being at regime at $t = 0$ means that at such time the derivative of the states must both be zeros:

$$\frac{d}{dt}\theta_o(0) = 0, \quad \frac{d}{dt}\theta_h(0) = 0 \quad (2.8)$$

Expanding the derivatives through their **ODEs** leads to the condition reported in the previous equation set. One can verify that setting the inputs to $\theta_a(t) = \theta_a, K(t) = K$, the **ODE** system admits a closed form solution that coincides with the functions of the Exponential Formulation. Hence, when the constant inputs assumption is verified the two formulations are identical.

Figure 2.1 represents the same set of differential equations in the visual language of the *System Diagram*. In particular, it highlights the feedback on the three state components, their reciprocal independence, and the fact that the only nonlinear operations are the functions K_x and K_y acting on the Load Factor.

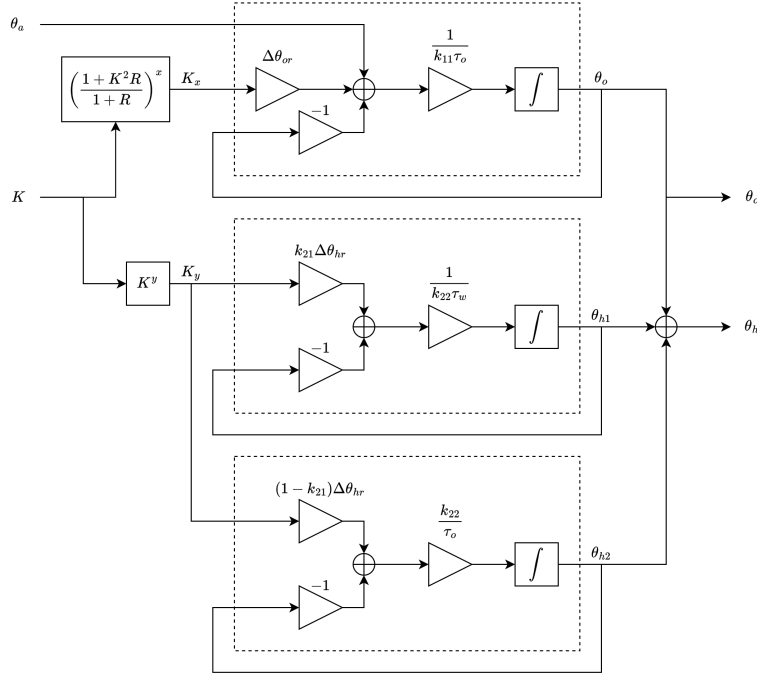


Figure 2.1: System Diagram of a Power Transformer as described by the IEC Model in the Differential Formulation

Loss of Life

The last element we need to analyze, is the relation between the Hotspot Temperature and the Loss of Life of a Power Transformer. In order to do so, we employ again the tool of the **ODE**, but without feedback. Hence, the Loss of Life is modeled as a function that starts at 0 and whose derivative depends on θ_h :

$$\begin{cases} \frac{d}{dt}L(t) = V(\theta_h(t)) \\ L(0) = 0 \end{cases} \quad (2.9)$$

The function V is called *Relative Aging Rate* and it depends on the physical and chemical properties of the insulation paper of the Power Transformer. There exist a great variety of possible models for the Aging Rate, some of which can even employ dedicated **ML** subsystems, but in the project the simple one presented in [3] is employed. The formulation assumes the paper can be either *Thermally Upgraded* or not, and defines a parameter free function for each of the two cases:

$$V(\theta_h) = \begin{cases} \exp\left(\frac{15\,000}{110+273} - \frac{15\,000}{\theta_h+273}\right) & \text{Thermally Upgraded Paper} \\ 2^{\left(\frac{\theta_h-98}{6}\right)} & \text{Non Th. Up. Paper} \end{cases} \quad (2.10)$$

2.2 Deterministic Control Theory

In the thesis, *Deterministic* Control Theory (**DCT**) indicates the algorithms that do not involve random processes. The techniques have the property that they always return the same set of output for the same set of inputs. On the contrary, *Stochastic* Control Theory employs random processes (e.g. the training of a **ML** algorithm) to produce results that may vary from on different executions.

State Space Representation

A pervasive tool used in **DCT** techniques is the State Space Representation (**SSR**) [6]. When Dynamical Systems are modeled through **ODEs**, their variables can be rearranged into a matrix representation that allows for a compact and standardized way to express them:

$$\text{SSR} : \begin{cases} \theta'(t) = A\theta(t) + Bu(t) \\ y(t) = C\theta(t) \end{cases} \quad (2.11)$$

Where $\theta(t)$ is the vector of system states while $\theta'(t)$ is its derivative. $u(t)$ is the vector of system inputs, and $y(t)$ the vector of system outputs. A , B and C are matrices embodying the coefficients of the **ODEs**; depending on the system they can be either constants or time varying, the report considers only the former case. The greatest advantage of **SSR** is that it provides a standard way of expressing the system, so that **DCT** algorithms needs to be formulated only on such representation. Chapter 4 presents how the differential formulation of the IEC Model has been converted to its **SSR** equivalent.

Euler Discretization

Physical systems are usually modeled through **ODEs**, which leads their states to be defined in the *continuous-time* domain. However, many simulation software and control algorithms require their variables to be expressed in *discrete-time*. It means that given a *Sampling Time* Δt , we are interested only

in the values assumed by the variables at the time instants $i\Delta t$, $i \in \mathbb{N}$. Hence, the functions $\theta(t), u(t), y(t)$ become *Time Series* θ_i, u_i, y_i . Another difference is that in continuous-time the goal is to determine the derivative of the state $\theta'(t)$, while in discrete-time we want a formula to obtain the current state θ_i given its previous sample θ_{i-1} and the current input u_i .

Many algorithms have been developed to convert a continuous-time system into a discrete-time equivalent that approximates its behavior [16]; the project employed a popular one called “Euler Discretization” [7]:

$$\text{Euler : } \begin{cases} \theta'_i = A\theta_{i-1} + Bu_i \\ \theta_i = \theta_{i-1} + \Delta t \theta'_i \\ y_i = C\theta_i \end{cases} \quad (2.12)$$

The fundamental assumption is that we consider the derivative in between two time samples $i-1$ and i to be constant and equal to θ'_i . Then, we can use the continuous-time formulae to obtain the derivative from the previous state θ_{i-1} and an input, which can be either u_{i-1} or u_i according to the convention (which one we consider to be the active one in between the samples). Once we have the derivative, since it is (considered) constant then its integral in between the samples is simply $\Delta t \theta'_i$, so it is the quantity we have to sum to θ_{i-1} to obtain the current state.

Luenberger Observer

A typical application of **DC**T is when we have a physical system that converts inputs $u(t)$ to outputs $y(t)$, but of which we do not know the internal state $\theta(t)$. Usually the model is run in parallel with the real system and fed with the same input; if the model is correct then its state is a good approximation of the real one. The only usage of the model output and the real output is to compare them in case we want to determine the model performance.

The Luenberger Observer [8] is a technique that allows us to actively exploit the knowledge of the real output to improve the state estimation rather than using it for comparison only. Retaining the symbol $y(t)$ for the model output and introducing $y^{\text{ref}}(t)$ for the *reference output*, i.e., the output of the physical system, the correction term $\theta^L(t)$ can be annexed a to the **SSR**:

$$\begin{cases} \theta'(t) = A\theta(t) + Bu(t) + \theta^L(t) \\ \theta^L(t) = L(y^{\text{ref}}(t) - y(t)) \\ y(t) = C\theta(t) \end{cases} \quad (2.13)$$

Such correction term is equal to the difference between the model output and the reference output, multiplied by a parameter matrix L , called *Luenberger Gain*. No further theoretical details are given on the gain, the selection of its value largely depends on the design choices of the system engineer. Chapter 4 explains the assumption made to determine the value of L for the project.

The Euler Discretization algorithm can be applied to systems empowered with the Luenberger Observer, giving the following discrete-time system as a result:

$$\begin{cases} \theta'_i = A\theta_{i-1} + Bu_i \\ \theta_i^L = L(y_{i-1}^{\text{ref}} - y_{i-1}) \\ \theta_i = \theta_{i-1} + \Delta t(\theta'_i + \theta_i^L) \\ y_i = C\theta_i \end{cases} \quad (2.14)$$

Kalman Filter

The Kalman Filter [9] is a powerful algorithm to improve the state estimation of a discrete-time system through the usage of a reference output. Compared to a Luenberger Observer, for complex systems it can give better estimations, at the price of a longer computational time. It assumes that the reference output is affected by a certain noise, due to measurement uncertainty, which is encoded in the *Measurement Covariance Matrix* R . Similarly, since the model is just an approximation of the real system also the passage from a sample $i-1$ to the following one i introduces uncertainty, encoded in the *Prediction Covariance Matrix* Q .

The Algorithm maintains the estimation of the system state in θ_i , but also propagates the uncertainty between time samples, through usage of the variable Σ_i . The two variables evolve according to two steps, called “Predict” and “Update” [17]:

$$\text{Predict : } \begin{cases} \theta'_i = A\theta_{i-1} + Bu_i \\ \hat{\theta}_i = \theta_{i-1} + \Delta t \theta'_i \\ \hat{y}_i = C\hat{\theta}_i \\ \hat{\Sigma}_i = Q + A\Sigma_{i-1}A^T \end{cases} \quad \text{Update : } \begin{cases} K_i = \hat{\Sigma}_i C^T (C\hat{\Sigma}_i C^T + R)^{-1} \\ \theta_i = \hat{\theta}_i + \Delta t K_i (y_i^{\text{ref}} - \hat{y}_i) \\ y_i = C\theta_i \\ \Sigma_i = \hat{\Sigma}_i - K_i C \hat{\Sigma}_i \end{cases} \quad (2.15)$$

The Prediction step, converts the previous sample θ_{i-1} to the estimated

current sample $\hat{\theta}_i$ through a direct application of the system dynamic, which happens to be an Euler Discretization. As anticipated the operation introduces uncertainty, so Σ_{i-1} is modified to incorporate the new noise Q , obtaining the estimated current uncertainty $\hat{\Sigma}_i$

Then, the Update step has the duty of using the reference to improve the estimation. First, $\hat{\Sigma}_i$ is used together with the Measurement Covariance R to compute the *Kalman Gain* K_i . The matrix is multiplied by the difference between the reference output and the model output, precisely as the Luenberger Gain, and the correction factor is summed to $\hat{\theta}_i$ to obtain the definitive value for the next sample θ_i . Such operation reduces the uncertainty, hence we need to update $\hat{\Sigma}_i$ as well generating Σ_i , the definitive uncertainty of sample i .

In practice the Kalman Filter can be considered like a time varying Luenberger Observer. Each iteration it computes the best gain by taking into account the uncertainties we have on both the model and the measurements.

2.3 Recurrent Neural Networks

RNNs are Machine **ML** designed to detect the patterns present into an input sequence [18]. Their most famous application embrace Language Modeling, Text Generation and Speak Recognition [10], but the project focuses on another widespread utilization of the tool: time series analysis.

Artificial Neural Network Formulation

RNNs are based on the Artificial Neural Network concept, a fundamental building block of **ML**. Following the notation introduced in [19], an ANN is a function that maps an input X to an output O , passing through an “hidden state” H . The conversion from X to H is given by a product with a parameter matrix, often followed by the addition of a bias, to which an *activation function* σ_h is applied. The state is then converted to the output following the same type of operations, but with different parameters and potentially a different activation function:

$$ANN : \begin{cases} H = \sigma_h (W_{xh}X + b_h) \\ O = \sigma_o (W_{ho}H + b_o) \end{cases} \quad (2.16)$$

Figure 2.2 highlights the information flow in an ANN. The learning process, also called Training, consists in finding the best values for the matrices and the biases so that the Network approximates the input-output relation of a

given dataset according to a comparison metric, which is usually called *Loss*. The algorithm traditionally employed for such purpose is the *Backpropagation* [20]; explaining the details under its functioning is beyond the purpose of the background section.

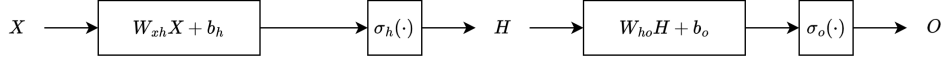


Figure 2.2: Scheme of an Artificial Neural Network

Recurrent Neural Network Formulation

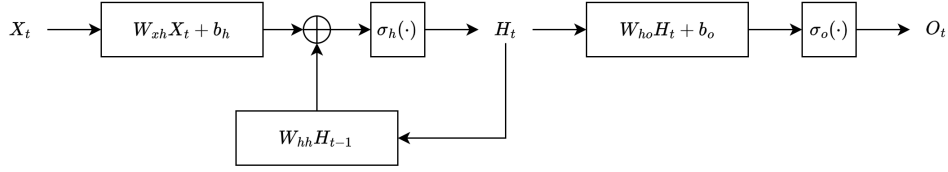
A **RNN** is an ANN that propagates information across time, because it retains the value of the state and uses it to enhance the computation of the next one [21]. The main assumption to the approach is the existence of a relation between two consecutive outputs, such as it happens in time series. From a practical point of view, it is achieved by adding a new term to the state computation, which includes a new parameter matrix to be multiplied by the previous state:

$$RNN : \begin{cases} H_t = \sigma_h (W_{hh}H_{t-1} + W_{xh}X_t + b_h) \\ O_t = \sigma_o (W_{ho}H_t + b_o) \end{cases} \quad (2.17)$$

Figure 2.3 shows the impact such decision has on the Network. Comparing it with Figure 2.2, we can notice that the only difference is the introduction of a feedback affecting H . It is the modification that allows **RNNs** to capture temporal dependencies in the input sequence, but also introduces new challenges in the training process. Again, the objective is to find the values of the parameters that minimize a Loss on a dataset, in which we assume the samples to be properly ordered. The modified version of Backpropagation typically used to train **RNNs** is called “Backpropagation Through Time” [10, 19].

Comparison With Control Theory

Starting from the **RNN** formulation and perform a very simple name change: we call the input X_t , the state H_t and the output O_t as one would do in control theory, *i.e.*, u_t , θ_t and y_t respectively. Then, we decide both the activation functions to be the identity. We get the following:

Figure 2.3: Scheme of a **RNN**

$$\begin{cases} \theta_t = W_{hh}\theta_{t-1} + W_{xh}u_t + b_h \\ y_t = W_{ho}\theta_t + b_o \end{cases} \quad (2.18)$$

From it, we can recognize the structure of a discrete-time Linear System. Hence, as also highlighted by [21], **RNNs** are models of dynamical systems, exactly as the one studied in Control Theory. The main difference, is that in **DCT** the matrices and the biases are obtained from the knowledge we have of the system, e.g. from the physical model or a known mathematical relation, while in **RNNs** they are the output of an optimization algorithm on the data, which often employ stochastic processes.

Hence, the advantage of a **RNN** over **DCT** is that it can be used also on systems of which we have no physical knowledge, but only raw pairs of input and outputs; it is known as *System Identification*[22]. The drawback is that it lacks interpretability: we do not know the relation between the system and the parameters, so for even a minimal design change we need to generate a new dataset and re-run the training entirely, while in **DCT** we usually know how changes to the real system impact the digital model.

2.4 Related Works

The problem of temperature estimation in Power Transformer is popular in the literature. The most important related work for the thesis is Tembo et al. [1], which employs three **ML** algorithms (namely Artificial Neural Networks, Time-series Dense Encoder and Temporal Convolutional Networks) to estimate the top-oil temperature and compare the accuracy of their results with the IEC 60076-7:2005 standard. Differences between the project and Tembo et al. are not limited to a dissimilar **ML** algorithm, the **RNN**, and the focus on hotspot temperature rather than the top-oil, but include also a different goal: the article aims to evaluate alternatives to the IEC Model, while the thesis aims to find valuable modifications to it.

The study by Bragone [4] is another example of ML-driven temperature estimation. It employs Physics-Informed Neural Networks to approximate the heat diffusion equation, which allows to estimate internal temperatures of a Power Transformer knowing the top-oil. Improvements to the thermal models have been also searched outside the ML domain; the work by Susa et al. [23] exploits the knowledge of the physical laws that govern the machine, such as the heat transfer theory, to formulate new computation method for the hotspot temperature.

Finally, it worth to mention benchmarking platforms like Gradnik et al. [24], which do not aim to introduce new models but to define pipelines for a pragmatic comparison of the already existing ones.

Chapter 3

Research Methodology

The chapter provides the details regarding the methodological framework that drove the conduct of the research. Section 3.1 presents and describes the Research Paradigm employed. Section 3.2 outlines the steps of the Research Process, while section 3.3 is dedicated to the dataset. Section 3.4 empowers the repeatability of the experiments, providing the Hardware and Software specifications. In section 3.5 and section 3.6 is described how the correctness of the results is checked and how they have been evaluated respectively. Section 3.7 formally defines the comparison metrics, while the system documentation is described in Section 3.8.

3.1 Research Paradigm

The project was conducted according to a *Quantitative* Paradigm, namely the *Experimental Research Method* [12]. According to the paradigm, the researcher design one or more experiments, whose outcomes can be quantified through numerical values. Then, a mathematical analysis can be carried out on such values, in such case a set of comparison metrics, and the numerical result of the analysis is used to derive the conclusions of the research. Such Approach involves both induction and deduction: the deductive component is embedded in the analysis step, where a set of known mathematical rules is used to process the data, while the inductive component is predominant in the conclusion step, where the outcomes are generalized to formulate new rules.

3.2 Research Process

The theoretical framework of the Experimental Research Method has been declined into a Research Process characterized by the following steps:

1. *Literature Study*: An extensive background research on Power Transformers have been carried out, employing scientific articles as well as consultancies with the specialized personal of Hitachi Energy Ltd.
2. *IEC Model Implementation*: Implement the IEC Model on python, following the specification prescribed by the international standard [3].
3. *IEC Model Evaluation*: Run the IEC Model on the dataset, CIGRE [24, 25] for the case of this project, and evaluate it with the desired performance metric.
4. *Improvement Loop*: Select an improvement technique, and perform on it the following steps:
 - (a) *Implementation*: Implement the selected algorithm.
 - (b) *Evaluation*: Run the new model on one or more datasets and the desired performance metric on the outcome.
 - (c) *Comparison*: Compare the performance of the new model to the one obtained by the unmodified IEC Model on the same dataset with the same metric.
5. *Conclusion*: Once you have performed the *Improvement Loop* many times, you can infer from the comparisons which techniques give a valuable performance improvement and which are negligible.

In addition, the student had the opportunity to visit the real Power Transformer plant in Västerås. It was organized only to maximize the learning experience, no physical simulation has been run on the machines. The visit needs to be considered as a component of the *Literature Study* step rather than the *Conclusion*.

The greatest advantage of the outlined Research Process relies in the enormous flexibility it provides regarding the algorithms and the performance metrics to be employed. The *Improvement Loop* have been run on a few possible techniques, but any researcher can execute the same exact workflow with their own model and performance metric: their results would be

immediately comparable to the one presented in the thesis, minimizing the effort required to continue the project and maximizing the potential innovation of the research.

3.3 The Dataset

For the project, the dataset presented in the CIGRE DTTM Benchmarking platform [24, 25] have been used, under suggestion from Hitachi Energy Ltd. It is an open-source dataset specifically designed for the evaluation of Power Transformers' models; it contains time series measurement for the most common Inputs and Outputs of a Power Transformer, as well as separate parameter files which collect the specific features of the individual machines the time series refer to. The files have been preprocessed to extract only the data useful for the IEC Model, as explained in Section 4.4.

Note that even though the platform [25] contains both data as well as benchmarking pipelines to analyze the results, the project did not use the latter. The reason, is that the research does not aim to be an instance of the CIGRE platform: it was commissioned by Hitachi Energy Ltd, then the nomenclature and performance indices suggested by them have been followed, even in case of incompatibility with the ones selected by the CIGRE group.

3.4 Testing Environment

Hardware Specifications

All the experiments have been run on a Lenovo Legion C7 personal computer. The specifications strings of its major components are reported in Table 3.1.

Component	Specification String
CPU	Intel(R) Core(TM) i7-10875H CPU @ 2.30 GHz
RAM	32,0 GB
GPU	Intel(R) UHD Graphics

Table 3.1: Hardware specifications of the computer used to run the simulations

Software Specifications

The entire project has been developed using Python, version 3.10.11. All the dependencies have been gathered in the appropriate `requirements.txt` file in the root folder of the project to guarantee repeatability, while in the section only the most important libraries are listed, together with the purpose they were used to.

NumPy: A numerical computation library especially suited for matrix operations.

pyTorch: ML library used to implement the different RNN architectures.

Pandas: A tool that helps you managing in-out files. It has been used it to perform the dataset manipulations.

Matplotlib: Popular graphical library used to produce scientific plots.

3.5 Quality Assurance

Validity of Method

The most crucial assumption of the research is that what is denoted by “IEC Model” in the thesis coincides with the model used in the industry and described in the international standard [3], without implementation errors. To check the validity of the statement, an additional experiment have been designed. Since the annex C of the standard [3] reports two tables showing the expected evolution of the hotspot temperature (θ_h) and the loss of life (L) with given inputs, the model have been run on the dataset: it obtained precisely the same outputs. Note that the check could not be done on the CIGRE dataset [24, 25] since it contains the values measured on the real Power Transformer rather than the output expected from the IEC Model.

Reliability of Method

The reliability of the method is directly linked with its repeatability. Not only the entire project source code is available on GitHub at [26], but also the selected parameters are available at the same location. In case of replication tentative all the deterministic techniques, *i.e.*, the Traditional Control Theory’s algorithms and the Evaluation Metrics, shall give the same exact results as the ones of the report.

On the contrary, the process of training a Neural Network such as a **RNN** is not deterministic but stochastic, which means it involves a certain amount of randomness. Then, running the training pipeline would produce models similar but not precisely identical to the ones obtained, as expected when dealing with **ML**.

Reliability of Data

To guarantee the reliability of the experiments the data preprocessing pipeline and the files resulting from it (See section 4.3) have been shared, together with a copy of the original data-files used in the GitHub project. Since the CIGRE dataset is licensed **GNU's General Public License (GNU-GPL)**, the files need to have the same license, on the contrary the other unrelated code components do not need to. It ensures the repeatability of the experiments and also respects the conditions of usage of the original dataset.

3.6 Evaluation Framework

Research Variables

An experiment of the project depends mainly on two concepts:

dataset it is a couple of two kinds of related measurements: the physical inputs to a Power Transformer system and the corresponding physical outputs.

$$\text{dataset}_d = (\text{input}_d, \text{output}_d)$$

model it is an algorithm implemented in `Python` that given the measurement of a Power Transformer input tries to approximate the respective output. Such approximation is called *prediction*.

$$\text{prediction}_{i,d} = \text{model}_i(\text{input}_d)$$

Analysis Technique

The conclusions of an experiment are drawn from a comparison between the prediction a model produces from a certain input and the output stored in the same dataset. The evaluation is obtained by the application of a comparison metric (see section 3.7) between the two:

$$\text{comparison}_{m,i,d} = \text{metric}_m(\text{prediction}_{i,d}, \text{output}_d)$$

The value encodes how much the model i is good at approximating the dataset d according to the performance metric m .

3.7 Comparison Metrics

The project generates outputs that are compared with their respective reference values. Since the analyzed system works in discrete-time, both the outputs and the references are *Time Series*, i.e., ordered sequences of values that represent the evolution of a variable. In the section, some Comparison Metrics commonly used when dealing with time series are defined and described. x and y are used as placeholders for any two generic time series, and the i^{th} elements of the sequences are indicated with x_i and y_i respectively. Since all the metrics employed are commutative, there is no need to specify which placeholder substitutes the output series and which one the reference series.

Mean Absolute Error

$$\text{MAE}(x, y) = \frac{1}{N} \sum_{i=1}^N |x_i - y_i| \quad (3.1)$$

The comparison metric computes the differences between correspondent elements in the time series and performs an average on their absolute values [13]. Its output is a positive scalar value, which is equal to 0 if and only if the two series are identical. It represents how much we expect the difference between two correspondent elements of the series to be, so the lower the MAE value the more x and y are similar time series.

Root Mean Square Error

$$\text{RMSE}(x, y) = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - y_i)^2} \quad (3.2)$$

The **RMSE** is similar to the **MAE**, but it computes the average on the square of the differences rather than their absolute values, and performs a square root at the end [13]. Consequently, a large difference between two elements is even more impactful after a square, which makes the metric much more sensible to outlier than the **MAE**; such feature can be desirable or not according to the situations [27], so the common practice is to present both **MAE** and **RMSE** to the reader when we do not make assumption on the error distribution [28].

It retains the property of returning a positive scalar value, equal to 0 only in case the series are identical. The interpretation of the metric is how much we expect the difference between two correspondent elements of the series to be, but giving more weight to eventual outliers. Again, the lower the **RMSE** the more x and y are similar.

Mean Square Error

$$\text{MSE}(x, y) = \frac{1}{N} \sum_{i=1}^N (x_i - y_i)^2, \quad \sqrt{\text{MSE}(x, y)} = \text{RMSE}(x, y) \quad (3.3)$$

The **Mean Square Error (MSE)** is the square of the **RMSE**, *i.e.*, a **RMSE** without the final square root [13]. The reason for the modification is that in both **ML** and Optimization Algorithms, comparison metrics can be employed as *Losses*, but the algorithm needs their derivative as well. Removing the square root from **RMSE** creates a function whose derivative is much more computationally efficient, while preserving the property that the output is 0 for and only for identical series.

The drawback is that the metric is much less interpretable than the other two, especially because it does not preserve the unit of measure of its inputs [13]. It means that if x and y have unit of measure $[u]$, **MAE** and **RMSE** give an output that has still unit of measure $[u]$, while **MSE** returns a value in the $[u^2]$ domain. However, analyzing its construction we can still conclude that the lower the **MSE** the more x and y are similar.

3.8 System Documentation

To maximize the repeatability, the entire code of the research had been made publicly available on GitHub at [26]. The repository includes all the Python sources, the preprocessed datasets used for the experiments and also some notable outputs obtained. A file called `requirements.txt` can be used to automatically install all the required libraries, while `README.md` describes the roles of each single file as well as the proper modalities of running the code. Finally, the project have been extensively documented with Doxygen, with the full System Documentation available at the GitHub-Page [29].

Chapter 4

System Design

The chapter presents how the background tools described in Chapter 2 have been employed to design the system explored in the project. Section 4.1 explains how the IEC Model have been converted to the SSR format, while Section 4.2 describes the Luenberger Gain computation. Section 4.3 is dedicated to the RNNs and their architectures. Section 4.4 provides details on the data preprocessing algorithm, and finally Section 4.5 puts everything together to give a panoramic view of the entire system.

4.1 From IEC to SSR

Section 2.1 presents the IEC Model formulated as a set of ODEs. To be able to simulate the system and empower it with DCT techniques, the first step is to convert the ODEs into a SSR. In order to do so, the State Vector and the Input Vector have been defined in terms of the IEC variables as following:

$$\theta := \begin{bmatrix} \theta_o \\ \theta_{h1} \\ \theta_{h2} \end{bmatrix}, \quad \theta' := \begin{bmatrix} \theta'_o \\ \theta'_{h1} \\ \theta'_{h2} \end{bmatrix}, \quad u := \begin{bmatrix} \theta_a \\ K_x \\ K_y \end{bmatrix} \quad (4.1)$$

The advantage of using K_x and K_y as inputs instead of directly the load factor is that since they capture already all the nonlinearities of the system then the remaining equations are linear and convertible into SSR. The disadvantage is that in a situation where we want to compute the input, then we would need additional steps to reconstruct K from them. However, the project does not deal with input optimization, we assume it to be given and unmodifiable.

The SSR arising from the vectors defined in Equation 4.1 is the following:

$$\text{IEC-SSR} : \begin{cases} \theta' = A\theta + Bu \\ \theta_o = C\theta \\ \theta_h = M\theta \end{cases} \quad (4.2)$$

The most important difference with the theoretical description in Section 2.2 is the presence of two different types of outputs:

Measurable Output $C\theta$: is the subset of the outputs that we can compare with a reference value measured on the real system. They are the ones we can use to improve the estimation in algorithms like the Luenberger Observer or the Kalman Filter. For the Power Transformer it coincides with the Top-oil Temperature θ_o .

Unmeasurable Output $M\theta$: is the subset of the outputs for which we do not have a reference during the simulation, but only at evaluation time. It is usually because they are very difficult to measure on the real system. For the Power Transformer it coincides with the Hotspot Temperature θ_h .

Given that the Top-oil is the first state, while the hotspot is equivalent to the sum of all states, the two output matrices are:

$$C = [1 \ 0 \ 0], \quad M = [1 \ 1 \ 1] \quad (4.3)$$

The coefficients of the **ODEs** are embedded in the matrices A and B . Due to the structure of the system they depend exclusively on constant parameters, hence they do not vary during the simulation. One can verify that the following matrices are the ones that make the **SSR** just defined in the section equivalent to the Differential Formulation presented in Equation 2.7:

$$A = \begin{bmatrix} \frac{-1}{k_{11}\tau_o} & 0 & 0 \\ 0 & \frac{-1}{k_{22}\tau_w} & 0 \\ 0 & 0 & \frac{-k_{22}}{\tau_o} \end{bmatrix}, \quad B = \begin{bmatrix} \frac{1}{k_{11}\tau_o} & \frac{\Delta\theta_{or}}{k_{11}\tau_o} & 0 \\ 0 & 0 & k_{21}\frac{\Delta\theta_{hr}}{k_{22}\tau_w} \\ 0 & 0 & (1 - k_{21})\Delta\theta_{hr}\frac{k_{22}}{\tau_o} \end{bmatrix} \quad (4.4)$$

Finally, the System requires an initial state $\theta(0)$, which can depend on the initial inputs. It is easily obtained by stacking the initial conditions of the Differential IEC Model:

$$\theta(0) = \begin{bmatrix} \theta_o(0) \\ \theta_{h1}(0) \\ \theta_{h2}(0) \end{bmatrix} = \begin{bmatrix} \theta_a(0) + \Delta\theta_{or}K_x(0) \\ k_{21}\Delta\theta_{hr}K_y(0) \\ (1 - k_{21})\Delta\theta_{hr}K_y(0) \end{bmatrix} \quad (4.5)$$

4.2 Luenberger Gain

In a Luenberger Observer, a crucial operation is the selection of the gain L . If its entries are too small then the convergence becomes slow, but if they are too large the system may encounter oscillatory behaviors. The best value is often decided with a trial and error on the simulation, but sometimes we have extra knowledge of the system's environment that can help us defining a plausible candidate value for it.

A parallelism with a Kalman Filter can be drawn, including the assumption that the Prediction Covariance Q and the Measurement Covariance R are known. Then, consider a Kalman Filter on which we have perfect knowledge of the initial sample, *i.e.*, $\Sigma_0 = 0$. Due to the formulae, the estimated uncertainty on the first sample is $\hat{\Sigma}_1 = Q$. The idea, is to take as Luenberger Gain the Kalman Gain of the first iteration, which according to the assumptions is:

$$L = K_1 = QC^T(CQC^T + R)^{-1} \quad (4.6)$$

The observer defined with the Gain is then a time-invariant approximation of a Kalman Filter, in which the time dependency is dropped by assuming perfect knowledge of the previous sample instead of propagating the uncertainty from an instant to the next one.

4.3 RNN Architecture

Selected Algorithm

Section 2.3 describes a **RNN** as a dynamical system in which the A and B matrices are computed from training on a dataset instead of given a priori. For the optimization process to work properly, the network shall not use exclusively the final state, but it shall consider its entire evolution through time. Such behavior is achieved by stacking all the computed samples θ_i into a unique simulation matrix Θ :

$$\Theta = [\theta_0 \quad \theta_1 \quad \dots \quad \theta_{T-1}] \in \mathbb{R}^{3 \times T} \quad (4.7)$$

Where θ_0 is the initial state, and each subsequent column is the result of applying the Euler Discretization of the system dynamic to the current one. Due to the matrix properties, the outputs do not need to be computed iteratively, but they can be obtained directly from Θ with a simple multiplication:

$$\Theta_h = [M\theta_0 \quad M\theta_1 \quad \dots \quad M\theta_{T-1}] = M\Theta \quad (4.8)$$

Provided the definitions, we are ready to comment on Algorithm 1, which enunciates the steps carried out by the RNNs.

Algorithm 1 RNN

Data: $\theta(0), u, A, B, \Theta_h^{\text{ref}}$

```

1 while Training Loop do
2    $A', B' \leftarrow \text{RNN.params}()$ 
3    $\Theta \leftarrow \text{Simulate}(\theta(0), u, A + A', B + B')$ 
4    $\Theta_h \leftarrow M\Theta$ 
5    $\varepsilon \leftarrow \text{Loss}(\Theta_h, \Theta_h^{\text{ref}})$ 
6    $\text{RNN.backprop}(\varepsilon)$ 
7 return  $A', B'$ 

```

Calling A' and B' the two matrix parameters of the network, they can be summed to the A and B ; the results are used to run a simulation which is stored in Θ . From the simulation, we can extract first the time series of Hotspot temperatures Θ_h , and then compare it to the reference Θ_h^{ref} to obtain the model loss ε . Finally, the Backpropagation-through-time algorithm uses the loss to update the model parameters, which are the matrices A' and B' . The process is repeated iteratively with the new parameters' values, until the model achieves the desired performance or an iteration limit is reached.

The fundamental innovation of such RNN Architecture with respect to traditional ones is that the parameters are first summed to the original IEC matrices, rather than returned directly, which means they embed the corrections to be applied to the current system knowledge. In other words, their representation is not a completely black box approach but it starts from the physical knowledge of the system and detect possible improvements to be applied to it, fusing ML with DCT into an hybrid approach.

Loss and Regularizers

Algorithm 1 introduces the concept of **ML** Loss. A Loss is a measure introduced to describe how good the model have been in describing the dataset for the current iteration; from its value the training algorithms derive the amount of correction to add to the **ML** parameters. Typically, it is the weighted sum of two components: A comparison Metric and a *Regularizer*. The comparison metrics, presented in Section 3.7 give a higher loss the more difference there is between the model prediction and the reference value. On the other hand, regularizers [30] are introduced to minimize the values of the parameters; it is desirable to reduce overfitting or when we do not want the model to produce excessive modifications to its input. Two common regularizers are the following:

$$L_1(p) = \lambda \frac{1}{N} \sum_{i=1}^N |p_i|, \quad L_2(p) = \lambda \frac{1}{N} \sum_{i=1}^N p_i^2 \quad (4.9)$$

Where p is the array of model's parameters and λ is an hyperparameter determining the relative importance of the regularizer with respect to the comparison metric. Due to the construction, L_1 is more prone to generate sparse parameters, i.e. the network has most weights near to zero and a few ones very high, while L_2 drives to have all the weights equally small in magnitude.

In the project **MSE** has been employed as the comparison metric, while the parameters are the entries of the matrices A' and B' , to which two different λ have been applied. The following two losses are obtained:

$$\begin{aligned} L_1(y, y^{\text{ref}}, A', B') &= \text{MSE}(y, y^{\text{ref}}) + \lambda_A \sum_{i,j} |A'_{i,j}| + \lambda_B \sum_{i,j} |B'_{i,j}| \\ L_2(y, y^{\text{ref}}, A', B') &= \text{MSE}(y, y^{\text{ref}}) + \lambda_A \sum_{i,j} A'^2_{i,j} + \lambda_B \sum_{i,j} B'^2_{i,j} \end{aligned} \quad (4.10)$$

Special emphasis should be put on the crucial role of the regularizers with respect to Algorithm 1: the higher λ_A , the more the training reduces the entries of A' , which represents the correction to A . Similar behavior for λ_B . Hence, higher λ s force the model to learn small correction matrices for IEC, while lower ones allow it to suggest substantial modifications. We can then to tune the importance of data fitting versus the modification's interpretability.

Batch Training

The management of the time series during training is an important design choice to be taken. The basic approach, called *Full-time Training*, is showed in Figure 4.1. The dataset is considered a monolithic block: the simulation starts at the beginning of the dataset, covers it entirely, and terminates at the last sample.

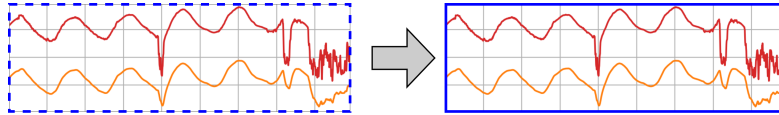


Figure 4.1: Full-time training

Figure 4.2 presents a different approach, called *Batch Training*. Many starting points are randomly sampled from the dataset and used to extract many shorter time series, called *batches*. As highlighted by the figure, it is possible for two batches to overlap, and there may also be some dataset portion not covered. Each batch undertakes its own simulation and gives a contribute to the loss.

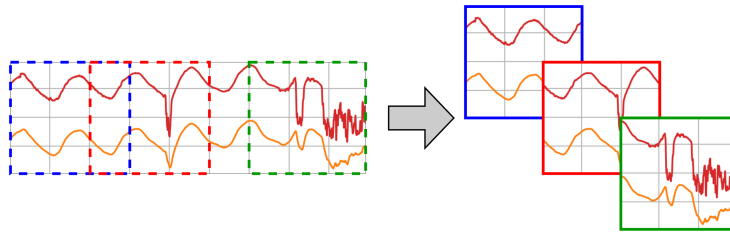


Figure 4.2: Batch training

The main advantage of Batch Training is that with enough batches the total time simulated is much longer than the full-time training, due to the numerous overlaps. Hence it is a form of *Data Augmentation* that reduces overfitting, especially when the batches are resampled each iteration of the Training Loop. Paradoxically, it can also result in a shorter training time: each batch can be simulated independently from the others, so we can exploit the *parallelization* features of `PyTorch` to process many of them simultaneously.

4.4 Data Preprocessing

The CIGRE dataset [24, 25] contains a vast number of time series from different Power Transformer machines. The thesis focuses on only one of them, namely the TTSa time series. It has been selected because among the ones including all the IEC parameters it is the longest. The data preprocessing consists in extracting only the inputs (t , K , θ_a) and the outputs (θ_o , θ_h) that we need, and saving them in CSV format. It is because some machines allow to measure additional inputs and outputs used by more complex models, but the project focuses on the IEC Model only. Additionally, some time series required data polishing, like removing corrupted rows or interpolate data gaps, but TTSa did not require any of them.

Regarding the extraction of the IEC parameters (presented in Table 2.1), the international standard [3] reports suggested values for many of them. In case of clash between the default values and the ones reported on CIGRE, the former ones are selected, to ensure the adhesion to the standard. Again, the extracted values are stored in a CSV file for easy retrieval.

Finally, when dealing with ML it is extremely important the distinction between *training* and *test* dataset. The Network optimizer is executed on the training dataset, where it aims to minimize the loss. The performance metrics need however to be measured on *unseen* data if we want to check the ability of the model to generalize; it is the purpose of the test dataset. For the DCT improvement no training is required, so their performance can be directly measured on the test dataset.

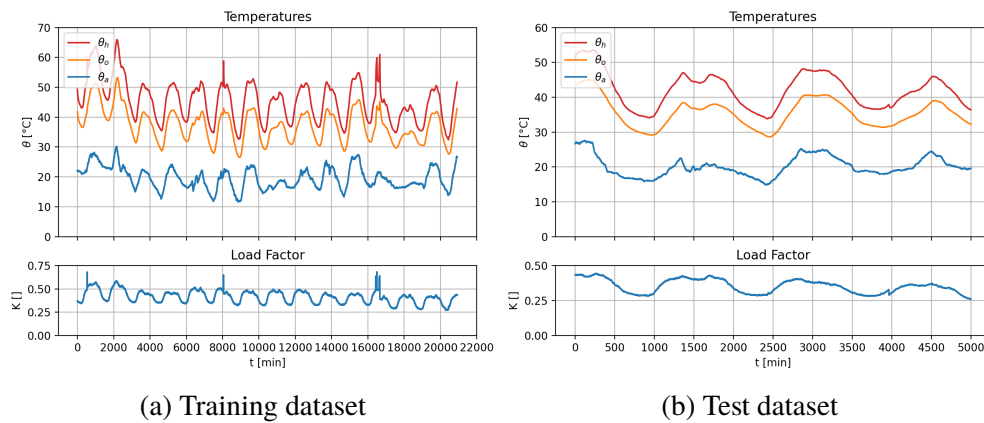


Figure 4.3: The dataset used for the experiments

In case of time series, the most practical way to achieve the training-test split is to select a time instant and consider all the samples preceding it as part of the train dataset, while the remaining are dedicated to test. Usually the train part shall cover from 70% to 80% of the data. Figure 4.3 presents a graphical visualization of the `TTSa` time series used in the project, already split between train and test.

4.5 Software Infrastructure

Simulation Routine

From a computational perspective, the project is organized into two main routines: `simulation` and `train`. The Simulation process starts by reading the CSV files of the IEC Model and converting them into NumPy arrays and matrices encoding the `SSR`, as explained in section 4.1. Then, it sums the matrices A' and B' to the system and the input matrix respectively. The improvement matrices can be set to zero when we want to simulate the baseline IEC Model, or they can be the output of the training process when we want to simulate an improved system.

When the required data has been loaded, the program allows to select the simulation algorithm; Three options have been implemented for it:

- Euler Discretization on the plain system
- Euler Discretization on the Luenberger Observer
- Kalman Filter (with Euler Discretization as `predict` function)

In the case of Luenberger, the gain is computed as explained in Equation 4.6. The outputs of the simulation are then compared with the reference ones according to the defined performance metrics, `MAE` and `RMSE`. Among the files stored by the routine, we find a CSV file for the output time series, different plots of the simulation, the comparison metrics (divided in a file for the temperatures and one for the loss of life) and a report of the final values of the matrices employed by the simulation. All the experiments on the test dataset have been run using the Simulation Routine.

Train Routine

The Train process is responsible for training the `RNNs`, and have been used exclusively on the train dataset. Like the other program the routine loads the

CSV files and converts them into the **SSR**, however the format is the one of the `PyTorch tensor`. Then, it applies Algorithm 1 on the system for many iterations. Most of the times the algorithm is executed on the batched dataset, as explained in section 4.3, but at some specific iterations it is executed on the full dataset with the goal of monitoring the performance. Finally, the routine returns the final values of A' and B' together with the final **MSE** they achieved on the dataset.

Name	A Mask	B Mask	Loss
RNN_a	$\begin{bmatrix} 0 & a & a \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	MSE
RNN_b	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	$\begin{bmatrix} b & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	MSE
RNN_tot_L1	$\begin{bmatrix} * & * & * \\ * & * & * \\ * & * & * \end{bmatrix}$	$\begin{bmatrix} * & * & * \\ * & * & * \\ * & * & * \end{bmatrix}$	L_1
RNN_tot_L2	$\begin{bmatrix} * & * & * \\ * & * & * \\ * & * & * \end{bmatrix}$	$\begin{bmatrix} * & * & * \\ * & * & * \\ * & * & * \end{bmatrix}$	L_2
RNN_p6	$\begin{bmatrix} a & b & 0 \\ 0 & 0 & c \\ 0 & 0 & d \end{bmatrix}$	$\begin{bmatrix} e & 0 & 0 \\ 0 & 0 & 0 \\ f & 0 & 0 \end{bmatrix}$	MSE
RNN_p5	$\begin{bmatrix} a & b & 0 \\ 0 & 0 & 0 \\ 0 & c & d \end{bmatrix}$	$\begin{bmatrix} e & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	MSE

Table 4.1: List of **RNN** Architectures tested in the project

As explained in Section 4.3, the greatest advantage of the approach is that we can force the improvement matrices to have the shape we desire. Table 4.1 reports the six architectures, where also the loss can be different. In the table entries marked with the same letters are forced to be the same, while entries set to zero are not modifiable during training. Matrices entirely marked by asterisks indicate that all entries are freely modifiable without constraints. The motivations behind the shape selection are presented in Section 5.2.

Chapter 5

Experiments and Results

The chapter is dedicated to the presentation of the experiments and their results. Further discussions are provided in Chapter 6. Section 5.1 compares the baseline IEC Model with the improvements from DCT. Section 5.2 describes the training of different RNN architectures, while Section 5.3 analyzes their performance on the test dataset to check if they can generalize what they learned during training. Section 5.4 presents the consequences of fusing ML and DCT approaches, and Section 5.5 shows how the properties of the hotspot temperatures are reflected to the loss of life.

5.1 Deterministic Improvements

The first experiment aims to measure the effects of the DCT improvements on the system. In order to do so, the simulation routine has been run on the test dataset with the improvement matrices set to zero, and selecting all the three different simulation algorithms. The performance metrics obtained are reported in Table 5.1, where the first row is the baseline IEC Model while the other two are the improvements.

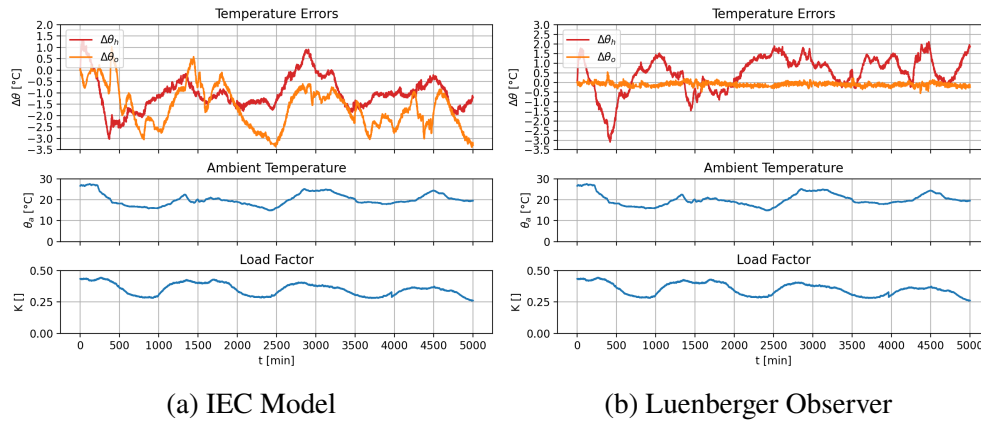
Looking at the total MAE, we can see that both the Luenberger Observer and the Kalman Filter give a net error reduction to the model, achieving a similar final error. The total RMSE behaves the same. Another feature shared by both the improvements, is that the error reduction over θ_o is much larger than the one on θ_h . It is expected, since both methods have direct access to the reference of top-oil, while the hotspot needs to be improved indirectly.

Model	MAE(θ_o)	MAE(θ_h)	MAE _{tot}
IEC Model	1.63 °C	1.12 °C	1.38 °C
Luenberger	0.12 °C	0.86 °C	0.49 °C
Kalman	0.10 °C	0.87 °C	0.48 °C

Model	RMSE(θ_o)	RMSE(θ_h)	RMSE _{tot}
IEC Model	1.85 °C	1.26 °C	1.58 °C
Luenberger	0.14 °C	1.03 °C	0.74 °C
Kalman	0.12 °C	1.04 °C	0.74 °C

Table 5.1: Detailed error metrics on the test dataset for the **DCT** improvements

Among the outputs of the experiment there is also Figure 5.1, which depicts the error evolution over time for the base model and the Luenberger observer; The Kalman Filter’s plot is not reported since it is almost identical to the one of Luenberger. From the plot is even clearer that the top-oil error is annihilated by the Observer, while the hotspot error is reduced but still present.

Figure 5.1: Errors on the test dataset for the **DCT** improvements, with Kalman Filter’s plot identical to Luenberger

5.2 RNN Training

In the second experiment the training routine has been run on the training dataset for all the six **RNN** architectures presented in Table 4.1. The goal is to determine which entries of the improvement matrices take the highest value,

and are consequently the most prone to influence the model behavior. Table 5.2 contains the **MSE** achieved by each network at the end of its training, together with the **MSE** of the baseline model for comparison. For the models that do *not* use regularizers, the value also corresponds to the Loss. In the following subsections we present the output of each **RNN** and explain the reason for their shape selection.

Architecture	MSE	Reduction
IEC Model	3.04 °C ²	baseline
RNN_a	1.59 °C ²	-47.70 %
RNN_b	1.48 °C ²	-51.32 %
RNN_tot_L1	1.06 °C ²	-65.13 %
RNN_tot_L2	1.08 °C ²	-64.47 %
RNN_p6	0.88 °C ²	-71.05 %
RNN_p5	0.89 °C ²	-70.72 %

Table 5.2: Error reductions on the train dataset for the **RNN** architectures

RNN_a

The network has a single parameter a in the A' matrix. One of the most important aspects of a Power Transformer neglected by the IEC Model is the presence of *convection*, a phenomena for which the oil of the hotspot rises to the top of the machine, influencing the top-oil. The positions in which a have been placed are the ones that allow the two hotspot components to influence the derivative of the top-oil. Consequently, set them to be different from zero may capture the effects of convection. The value of a found with the experiment is the following:

$$a = +2.032e-3$$

RNN_b

The second architecture comes from a consideration on in Figure 5.1 (a), which reports a simulation of the baseline IEC Model. Looking at it we can observe that both the errors are mostly below zero: it means the model tends to underestimate the temperatures. A possible countermeasure is an architecture that gives the input temperature (*i.e.*, the ambient) an higher impact on the

derivative of the top-oil. The training returned a positive value for b , which translates to a magnitude augmentation for the indicated matrix cell:

$$b = +8.470\text{e-}4$$

RNN_tot_L1

After having tried the two handcrafted improvements described before, we want to find an automatic way to identify the most sensible cells of the improvement matrices. The idea is to create an architecture able to fully modify all the parameters, but equipped with a loss that penalizes excessive improvement. The resulting network improves only the most sensible cells, leaving the less important ones with a value closer to zero. RNN_tot_L1 employs an L_1 regularizer and generates the following result, where values greater than 10^{-3} are enclosed in a square:

$$A' = \begin{bmatrix} \boxed{+1.712\text{e-}3} & \boxed{-1.663\text{e-}3} & -1.015\text{e-}4 \\ -5.041\text{e-}5 & +7.501\text{e-}4 & \boxed{-1.418\text{e-}3} \\ -6.884\text{e-}4 & +8.722\text{e-}4 & \boxed{-6.490\text{e-}3} \end{bmatrix}$$

$$B' = \begin{bmatrix} \boxed{-1.453\text{e-}3} & -3.953\text{e-}5 & -4.819\text{e-}5 \\ -7.052\text{e-}5 & -2.063\text{e-}5 & +6.172\text{e-}5 \\ \boxed{-1.960\text{e-}3} & +2.547\text{e-}5 & -1.378\text{e-}5 \end{bmatrix}$$

RNN_tot_L2

The network is very similar to the previous, as it has full control over all the eighteen entries of the improvement matrices and have a loss that penalizes excessive modifications. The difference is that such penalization is achieved with an L_2 loss rather than an L_1 . A single variation is sufficient to produce a completely different result:

$$A' = \begin{bmatrix} \boxed{+1.289\text{e-}3} & \boxed{-1.056\text{e-}3} & -5.994\text{e-}4 \\ +3.525\text{e-}4 & +7.384\text{e-}4 & -6.465\text{e-}4 \\ -4.193\text{e-}4 & \boxed{+1.333\text{e-}3} & \boxed{-1.617\text{e-}3} \end{bmatrix}$$

$$B' = \begin{bmatrix} \boxed{-1.266\text{e-}3} & -3.869\text{e-}5 & -5.597\text{e-}5 \\ +2.675\text{e-}5 & -1.060\text{e-}5 & -7.567\text{e-}6 \\ -8.321\text{e-}4 & -1.378\text{e-}5 & -2.175\text{e-}6 \end{bmatrix}$$

RNN_p6

Looking at the result of RNN_tot_L1 , 10^{-3} is selected as the threshold value to determine whether a cell is valuable or negligible. The RNN_p6 architecture has a structure corresponding to the cells of RNN_tot_L1 's output greater in magnitude than 10^{-3} , *i.e.*, the ones marked with a square. However, since no regularizer is employed the network can modify its entries without restrictions. The result obtained by such **RNN** has 6 parameters and is the following:

$$A' = \begin{bmatrix} +2.503e-3 & -2.383e-3 & 0 \\ 0 & 0 & -1.228e-2 \\ 0 & 0 & -9.042e-3 \end{bmatrix} \quad B' = \begin{bmatrix} -2.173e-3 & 0 & 0 \\ 0 & 0 & 0 \\ -4.998e-3 & 0 & 0 \end{bmatrix}$$

RNN_p5

The same procedure can be applied to the output of RNN_tot_L2 : checking the cells and placing the parameters on the ones higher than 10^{-3} , the structure of RNN_p5 is obtained. There are 5 valuable parameters in the obtained network, and their value after training is the following:

$$A' = \begin{bmatrix} +2.694e-3 & -1.278e-3 & 0 \\ 0 & 0 & 0 \\ 0 & -3.721e-3 & -9.888e-3 \end{bmatrix} \quad B' = \begin{bmatrix} -3.660e-3 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

5.3 Generalization Capabilities of RNNs

After having found the values of many improvement matrices through training with the previous experiment, it is time to evaluate their performance on the test dataset, to identify the most effective one. The **MSEs** already presented in Table 5.2 shall *not* be regarded as performance indices, since they were measured on the train dataset. The experiment is executed running the simulation routine on the test dataset, using the values of A' and B' proposed by the training of the different **RNNs**. Their performance have been recorded and reported in Table 5.3, together with the metrics of the baseline IEC Model for comparison.

We can see that all the models caused an improvement respect to the baseline on both **MAE** and **RMSE**, with RNN_p6 being the best one according

Architecture	MAE	Reduction	RMSE	Reduction
IEC Model	1.38 °C	baseline	1.58 °C	baseline
RNN_a	0.83 °C	-39.86 %	1.03 °C	-34.81 %
RNN_b	0.75 °C	-45.65 %	0.95 °C	-39.87 %
RNN_tot_L1	0.78 °C	-43.48 %	0.96 °C	-39.24 %
RNN_tot_L2	0.77 °C	-44.20 %	0.96 °C	-39.24 %
RNN_p6	0.72 °C	-47.83 %	0.87 °C	-44.94 %
RNN_p5	0.78 °C	-43.48 %	1.06 °C	-32.91 %

Table 5.3: Error metrics and error reductions on the test dataset for the RNN architectures

to both metrics. It is also notable to point out that RNN_p5, which has a very similar MSE during training, presents on the contrary the highest RMSE and a relatively high MAE. It signifies that the network *overfitted* the training dataset, since it has low generalization capabilities.

Another important aspect is highlighted by Figure 5.2, that plots the error evolution for the RNN_p6 and RNN_p6 networks. Looking at them we can see that, unlike what happens for DCT, the RNNs homogeneously reduced both the errors at the same time instead of privileging the top-oil error.

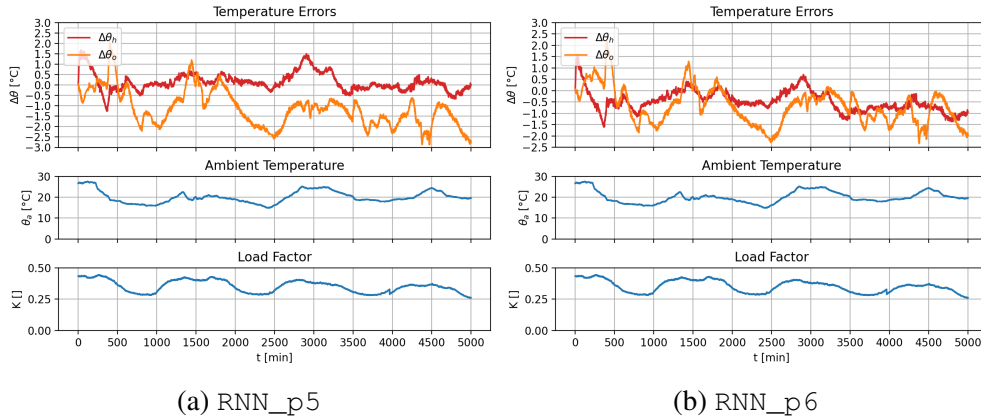


Figure 5.2: Errors on the test dataset for two representative RNNs

5.4 A Hybrid Approach

The goal of the experiment is to improve the IEC Model using **DCT** together with **ML**. According to Section 5.3 **RNN_p6** turned out to be the best **RNN**, hence it is the only one considered by the experiment. The simulation routine is run on the test dataset using **RNN_p6**'s improvement matrices together with the simulation algorithms of the **DCT** improvements, creating a *hybrid* approach. Table 5.4 reports the performance metrics for the new simulations and also the ones of the same techniques when used separately.

Model	MAE	Reduction	RMSE	Reduction
IEC Model	1.38 °C	baseline	1.58 °C	baseline
Luenberger	0.49 °C	-64.49 %	0.74 °C	-53.16 %
Kalman	0.48 °C	-65.22 %	0.74 °C	-53.16 %
RNN_p6	0.72 °C	-47.83 %	0.87 °C	-44.94 %
RNN_p6 + Luen.	0.34 °C	-75.36 %	0.54 °C	-65.82 %
RNN_p6 + Kalman	0.34 °C	-75.36 %	0.54 °C	-65.82 %

Table 5.4: Error metrics and error reductions on the test dataset for the hybrid improvements

We can see is the **DCT** improvements are more effective than **RNNs** in reducing the error, with a **MAE** reduction around -65% versus the -48% of **RNN_p6** and a **RMSE** reduction of -53% versus -45%. However, when we employ both of them we can reach the result of -75% on **MAE** and -66% on **RMSE**, the best performance gains among all the experiments. It is also important to notice that in the hybrid approach as well there is very little difference between the Kalman Filter improvement and the Luenberger Observer.

To understand why the hybrid approach is the best we can look at Figure 5.3, that reports the simulation plot of the baseline model, the Luenberger Observer, the selected **RNNs** and their combination. First, we compare the Luenberger with the baseline; as already emphasized such improvement mainly acts on θ_o , which perfectly sticks to the reference value, but leaves a lot of margin for θ_h . On the contrary, comparing the baseline with the **RNNs** both the outputs present a better behavior, but none of them precisely overlap with its reference. The hybrid approach combines the advantages of both methods: on the one hand, the error on θ_o vanishes due to the Observer, while on the other hand the error on θ_h is heavily reduced given the presence of the **RNN**.

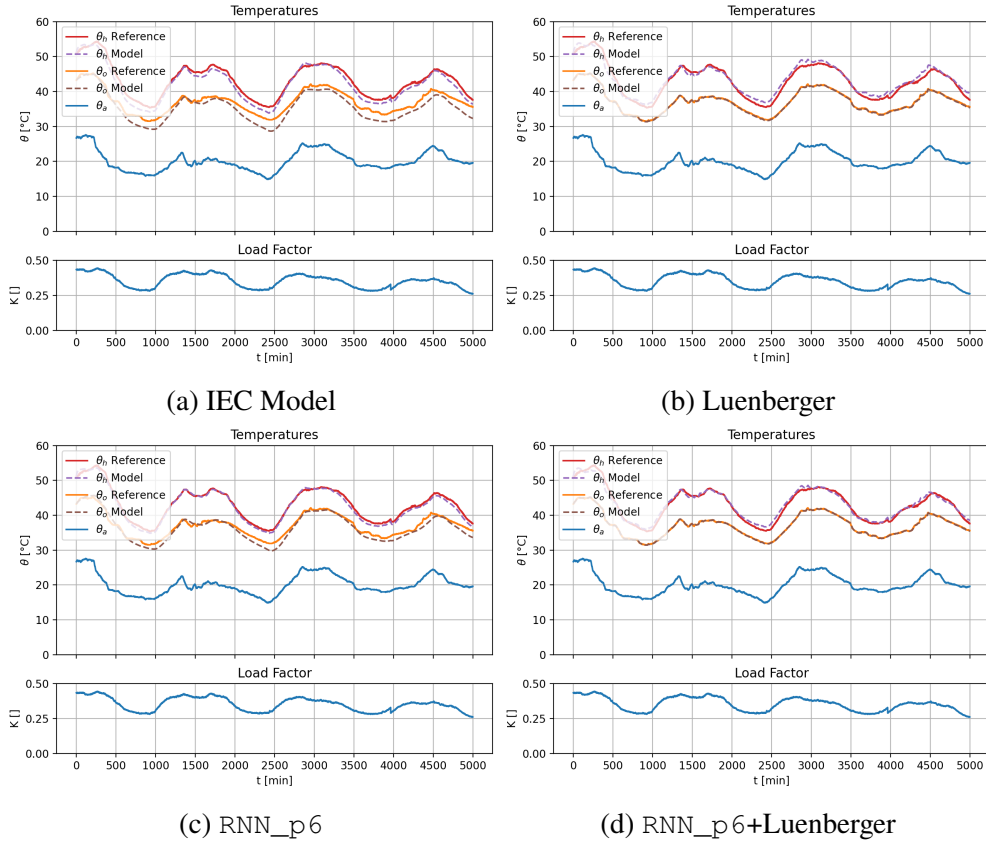


Figure 5.3: Simulations on the test dataset with hybrid improvements

5.5 Loss of Life

When an experiment is executed it does not just report the performance metrics relative to the temperatures, but also the ones regarding the loss of life. As discussed in Section 2.1, L is the integral of an exponential function of the hotspot temperature, hence little improvements in θ_h can translate into major improvements for L . Table 5.5 reports the MAE on the loss of life for the hybrid approach experiments. The column ΔL refers to the difference between the final value of L predicted by the model and the final reference value of L , which is 1.60 min.

As expected, the MAE on the loss of life is reduced even more than the one on the temperatures, with a -80% respect the baseline for the hybrid approach. The final value of L is an important metric, since it is the variable triggering the maintenance actions, and with the hybrid approach we are able to reach a -0.01 min difference respect the reference, versus a -0.17 min with

Improvement	MAE(L)	Reduction	Final ΔL	Reduction
IEC Model	0.10 min	baseline	-0.17 min	baseline
Luenberger	0.03 min	-70.00 %	0.06 min	-64.71 %
Kalman	0.03 min	-70.00 %	0.06 min	-64.71 %
RNN_p6	0.03 min	-70.00 %	-0.07 min	-58.82 %
RNN_p6 + Luen.	0.02 min	-80.00 %	-0.01 min	-94.12 %
RNN_p6 + Kalman	0.02 min	-80.00 %	-0.01 min	-94.12 %

Table 5.5: Loss of life metrics on the test dataset for the hybrid improvements

the unmodified IEC. Figure 5.4 displays the loss of life evolution for the IEC Model and the hybrid approach; The errors on the metric are cumulative due to the integral.

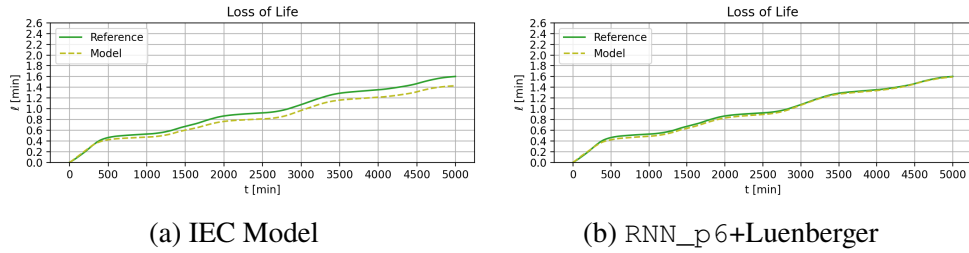


Figure 5.4: Loss of Life on the test dataset for the original and hybrid model

Chapter 6

Discussions

The chapter discusses the results presented in Chapter 5 and examines their consequences on the system. Section 6.1 describes the similarities and differences between the two DCT improvements. Section 6.2 compares the properties of the RNNs architectures, while Section 6.3 comments on the possible overfitting by some of them. Section 6.4 is dedicated to understanding the feasibility of the hybrid approach, and Section 6.5 explains to what extent the results obtained for the hotspot temperature can be transferred to the loss of life.

6.1 Luenberger vs Kalman

The first experiment ensured the equivalence between the Kalman Filter and the Luenberger Observer with the Gain described in Section 4.2. The experiment reveals they have a very similar performance on both MAE and RMSE, while their plots are almost indistinguishable. It means the Observer accomplishes its goal of behaving like a time invariant approximation of the Kalman Filter.

Since the Luenberger Observer is easier to implement than the Kalman Filter, a valuable conclusion is that the former is enough to guarantee a notable improvement to the system while minimizing the effort to implement it. Therefore, the Luenberger Observer is the best representative of DCT improvements for what concerns the research answer.

6.2 A Two-stages Approach to RNNs

The goal of the second experiment is to present a *modus operandi* to determine the most sensible cells of the improvement matrices. The information can be used to act only on such entries, maximizing the improvement with minimal model changes. The proposed approach consist in using first a network with full matrices but a Loss that penalizes changes and subsequently another one with constrained matrices but unconstrained loss.

First of all, the regularizers are a proper way to penalize changes, since both `RNN_tot_L1` and `RNN_tot_L2` have matrices whose values are much lower in magnitude compared to their counterparts `RNN_p6` and `RNN_p5`. On the other hand, removing the regularizer once fixed the final structure is essential to achieve non-negligible improvements, as witnessed by the fact that `RNN_p6` and `RNN_p5` achieve a better **MSE** respect the other two even if they have way less degrees of freedom.

An open point in the experiment is the definition of a pragmatic way to select the threshold to consider an improvement in a cell non-negligible. For the experiment the solution is handcrafted but future studies may design a systematic way to obtain the best structure to use for the second network.

6.3 The Problem of Overfitting

The third experiment is designed to check whether the **RNNs** learned valuable improvements or just overfitted the training dataset, by checking their performance on unseen data. What emerges from the simulation is that `RNN_p6` has the best generalization capabilities, since it couples a good **MSE** on the training dataset with low **MAE** and **RMSE** on the test dataset. On the contrary, `RNN_p5` has one of the lowest **MSE** during training but is not equally effective during test. Hence, we can deduce that some **RNNs** have better generalization capabilities than others.

It is important to notice that the overfitting of `RNN_p5` is not necessarily due to the structure of its improvement matrices, but to the training process itself. the reason can be a wrong selection of the learning rate, or an excessive number of iterations, or just an unfavorable batch sampling. Investigating properly the cause of overfitting is not the goal of the research, hence the procedure have been limited to train many networks types and pick the best one, which results to be `RNN_p6`.

6.4 Feasibility of the Hybrid Approach

The fourth experiment is the core of the project, and aims to demonstrate that **DCT** improvements and **ML** improvements can be employed together on the same system, without one compromising the effects of the other. When used together, the two approaches produce an error reduction much higher compared to when they are used independently. Furthermore, the presence of the **RNN**'s improvement does not cancel the equivalence between the Luenberger Observer and the Kalman filter, which on the opposite continue to have present similar performances. It is consequently possible to identify the best improvement found by the experiments as the hybrid approach Luenberger + RNN_p6.

The reason why the two approaches can work together is that they act on different aspects of the model: The **RNN** improves the system matrices, which means it aims to introduce new parameters or tune the existing ones, while the Luenberger Observer exploits the knowledge of the reference output to introduce a new input, which is the error on the previous sample. Hence, the changes brought by one approach do not interfere with the domain of the other and the improvements become cumulative.

6.5 Improvements on the Loss of Life

The hotspot temperature is directly related to the loss of life of the Power Transformer, hence the improvements found during the experiments bring beneficial consequences to the L variable as well. However, a notable bias we have to consider is that the **TTSa** time series operates on relatively low temperatures. The peak of 70°C reached by the dataset is far from the values that cause serious degradation to the machine, consequently the loss of life presents small values. In fact its reference final value is just 1.60 min over a simulation that covers around three days.

It means errors in the high temperatures have heavier consequences on the loss of life compared to the ones lower temperatures, a situation not confirmed yet by the experiments since they did not encounter high temperatures. Checking how the improvements affect the loss of life during the high temperatures can be a possible future work.

Chapter 7

Conclusions and Future Works

The chapter finalizes the thesis condensing the most important outcomes and indicating the possible directions to continue the work. Section 7.1 summarizes the conclusions of the project and its main contributions. Section 7.2 provides an answer to the research question, while Section 7.3 recalls the limited scope of the project. Section 7.4 presents different ways in which the research can be expanded, and finally Section 7.5 reports the personal reflections on the thesis.

7.1 Conclusions

The project explores the two fields of **DCT** and **ML**, using them independently as well as together to improve IEC Model in the temperature estimations of Power Transformers. Regarding the **DCT** field, the research highlights that with an accurate selection of the Gain, the Luenberger Observer can achieve the same performance improvement of the Kalman Filter with a considerably lower implementation effort. On the other hand, the **ML** field have been focused on **RNNs** and presented a way to first identify the most sensible matrix entries of a system and then improve them to better fit a training dataset.

When combined together into an hybrid approach the two domains obtained even better results, proving they can be implemented on the same system without one shadowing the other or clashing with it. Many points remain open, including finding an automatic approach to whether a matrix modification is significant or negligible, test the model with temperature closer to the security limit, and most importantly give a physical interpretation to the matrix changes proposed by a **RNN**.

7.2 Research Answer

Now that the experiments have been presented, analyzed and discussed we can give an answer to the Research Question presented in Section 1.2:

The Luenberger Observer, the Kalman Filter and the Recurrent Neural Network all proved to give significant performance improvements to the 'IEC Model' regarding the estimation of Power Transformers' hotspot temperature on historical datasets with time-varying load factor.

The hybrid approach consisting of a Recurrent Neural Network combined with either of the two deterministic algorithms emerged as the technique able to obtain the lowest error.

7.3 Limitations

The most important limitation of the research is that it deals with improvements to the IEC Model only. No other models have been taken into account, neither for improvement nor for performance comparison. Furthermore, the only DCT techniques addressed by the project have been the Luenberger Observer and the Kalman Filter, while the only ML algorithm have been the RNN architecture described in Section 4.3. No conclusion can be drawn for any method other than the three of them. It is also important to notice that the experiments have consisted in simulations on historical datasets, hence the physical Power Transformer Machines have not been involved in the process.

Regarding the limited scope of the conclusions, a personal note is that the Luenberger Observer and the Kalman Filter happened to be equivalent only in the specific case of Power Transformers, but they shall not be considered the same for the general case of all dynamical systems. Finally, in the training of RNNs no explicit action have been taken to contrast overfitting, except training many networks and checking which ones presented such problem.

7.4 Future Works

The project introduces the goal of improving the IEC Model of Power Transformer with the aid of DCT as well as ML, and implements a few techniques in such direction. To do so, it lays down the basis for different other projects that can contribute accomplishing similar goals; some notable possibilities are reported in the following list.

- *Simulate the system at high temperatures:* As explained in section 6.5, high temperatures make estimation errors more impactful on the loss of life. It can be valuable to see if the improvements used so far maintain their effectiveness at high temperatures or they become negligible.
- *Execute the algorithms on a physical machine:* For now the experiments have been digital simulations on a dataset, but the real machines can present additional behaviors. At a certain point, it is necessary to check whether the physical Power Transformers respond like their digital twin or not.
- *Determine the improvement threshold:* In the experiment presented in Section 6.2 the final structure of the improvement matrix have been selected using a threshold manually crafted looking at the results of the previous network. The approach can not be automatized, so it is necessary to design a systematic way to identify the most sensible cells from the output of the network with the regularizer.
- *Test more precise discretization techniques:* Since the system has been implemented from scratch, a simple discretization technique has been employed, which is the Euler Discretization. It may be worthy to employ more precise simulation algorithms, to verify whether they are able to reduce the numerical noise due to integration. A notable candidate for the purpose is the Runge Kutta family [31, 32].
- *Simulate in continuous-time:* Complementarily to the previous suggestion, one can remove entirely the discretization by simulating and optimizing the system in the continuous-time directly. The work on “Neural ODE” [33] presents a ML approach to optimize the parameters of an ODE, and can be of major help.

There is also the straightforward approach of implementing and testing new DCT and ML algorithms to discover even better improvements. For the particular case of exploring other RNN architectures, some suggestions are given in the following list.

- *Learn A' and B' from the parameters:* The project employs the output of the RNNs to improve the system matrices, but does not give a physical interpretation to the modifications. A step towards the ambitious goal can be to implement a network that obtains A' and B' explicitly embedding the parameters of Table 2.1 as well. It can help

identifying the parameters that influence each cell, helping formulating an interpretation for the changes proposed by the algorithm.

- *Contrast Overfitting*: Section 7.3 reports that the overfitting problem has not been addressed. To be precise, Batch Training can be considered a method to prevent overfitting, but the results show the necessity to implement additional techniques to mitigate the problem as much as possible.
- *Learn the Luenberger gain*: Selecting the Luenberger gain as an approximation of a Kalman Filter gives effective results, but maybe there can be even more appropriate values. ML can help, since we can embed the gain among the network parameters and optimize it through training a well. It would be of particular interest to check whether the optimal gain would be similar or radically different than the Kalman-like one.
- *Optimize on the Loss of Life*: The current implementation aims to minimize the training error on θ_o and θ_h . The hotspot value in particular is strongly related to the loss of life, but there may be situations in which the precision of θ_h does not imply the one of L . To check the eventuality, a RNN that backpropagates the error on the loss of life rather than the one on the temperatures one can implemented.

7.5 Reflections

The research explores a possible range of techniques to improve the estimation of the Hotspot Temperature in Power Transformers, especially when there is a time-varying Load Factor. Having an accurate value for the Hotspot Temperature is essential to have an accurate value for the loss of life of the machine, since higher temperatures cause higher degradation to the system. It can help companies such as Hitachi Energy Ltd to develop a better maintenance schedule for their plants, and therefore a better resource utilization with less wastes.

It is especially true considering that Renewable Energy resources produce time-varying loads rather than constants, then the thesis can be helpful for companies to handle the ecological transition without losing efficacy in their maintenance flow. For the aforementioned reasons, the research contributes to the UN SDGs [11] number 7 “Affordable and Clean Energy” and number 9 “Industry, Innovation and Infrastructure”.

The work is not limited to the industrial context, but it presents scientific relevance as well. Its main contribution is the direct comparison it offers between **DCT** and **ML**, which culminates into an hybrid approach exploiting the advantages of both. Under such perspective, Power Transformer have been a particular system instance on which the two domains could be tested, but the methodological flow that emerged can easily be adapted to many systems structures typical of Automatic Control.

References

- [1] F. Tembo, F. Bragone, T. Laneryd, M. Barreau, and K. Morozovska, “Data-driven vs traditional approaches to power transformer’s top-oil temperature estimation,” *Results in Engineering*, vol. 30, p. 110645, 2026. doi: <https://doi.org/10.1016/j.rineng.2026.110645>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2590123026016798>
- [2] P. H. Pietrani, “Explosion d’un transformateur: les salariés de rte alertent sur l’état du réseau électrique français,” *Révolution Énergétique*, July 24, 2026. [Online]. Available: <https://www.revolution-energetique.com/actus/explosion-dun-transformateur-les-salaries-de-rte-alertent-sur-let-at-du-reseau-electrique-francais>
- [3] I. I. Standard, “Power transformers - part 7 - loading guide for oil-immersed power transformers,” IEC: Geneva, Switzerland, December 2005, updated in 2018, so marked as Withdrawn.
- [4] F. Bragone, “Physics-informed neural networks and machine learning algorithms for sustainability advancements in power systems components,” Ph.D. dissertation, KTH Royal Institute of Technology, 2023.
- [5] H. Nordman and M. Lahtinen, “Thermal overload tests on a 400-mva power transformer with a special 2.5-pu short time loading capability,” *IEEE transactions on power delivery*, vol. 18, no. 1, pp. 107–112, 2003.
- [6] D. Rowell, “State-space representation of lti systems,” *MIT*, pp. 1–18, 2002. [Online]. Available: <http://web.mit.edu/2.14/www/Handouts/StateSpace.pdf>
- [7] E. Kreyszig, *Advanced Engineering Mathematics*. John Wiley & Sons, Inc., 2011. ISBN 978-0-470-45836-5

- [8] D. Luenberger, "Observers for multivariable systems," *IEEE transactions on automatic control*, vol. 11, no. 2, pp. 190–197, 1966.
- [9] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 03 1960. doi: 10.1115/1.3662552. [Online]. Available: <https://doi.org/10.1115/1.3662552>
- [10] R. M. Schmidt, "Recurrent neural networks (rnns): A gentle introduction and overview," *arXiv preprint arXiv:1912.05911*, 2019.
- [11] U. N. D. of Economic and S. Affairs, "The sustainable development goals," 2015. [Online]. Available: <https://sdgs.un.org/goals>
- [12] S. Sukamolson, "Fundamentals of quantitative research," *Language Institute Chulalongkorn University*, vol. 1, no. 3, pp. 1–20, 2007.
- [13] C. Lorenzo, A. Elwood, M. Leonardi, A. Mohamed, and R. Alessandro, "A survey and taxonomy of loss functions in machine learning," *AI*, vol. 7, no. 4, p. 128, 2026.
- [14] H. Nordman, N. Rafsback, and D. Susa, "Temperature responses to step changes in the load current of power transformers," *IEEE Transactions on Power Delivery*, vol. 18, no. 4, pp. 1110–1117, 2003.
- [15] B. of indian standards, "Indian standard on power transformers - part 7 - loading guide for oil-immersed power transformers," October 2009. [Online]. Available: <https://law.resource.org/pub/in/bis/S05/is.2026.7.2009.pdf>
- [16] M. F. Fathoni and A. I. Wuryandari, "Comparison between euler, heun, runge-kutta and adams-bashforth-moulton integration methods in the particle dynamic simulation," in *2015 4th International Conference on Interactive Digital Media (ICIDM)*. IEEE, 2015, pp. 1–7.
- [17] G. Welch, G. Bishop *et al.*, "An introduction to the kalman filter," 1995.
- [18] L. R. Medsker, L. Jain *et al.*, "Recurrent neural networks," *Design and applications*, vol. 5, no. 64-67, p. 2, 2001.
- [19] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola, *Dive into deep learning*. Cambridge University Press, 2023.

- [20] S.-i. Amari, “Backpropagation and stochastic gradient descent method,” *Neurocomputing*, vol. 5, no. 4-5, pp. 185–196, 1993.
- [21] H. Salehinejad, S. Sankar, J. Barfett, E. Colak, and S. Valaee, “Recent advances in recurrent neural networks,” *arXiv preprint arXiv:1801.01078*, 2017.
- [22] J. G. Kuschewski, S. Hui, and S. H. Zak, “Application of feedforward neural networks to dynamical system identification and control,” *IEEE transactions on control systems technology*, vol. 1, no. 1, pp. 37–49, 1993.
- [23] D. Susa, M. Lehtonen, and H. Nordman, “Dynamic thermal modelling of power transformers,” *IEEE transactions on Power Delivery*, vol. 20, no. 1, pp. 197–204, 2005.
- [24] T. Gradnik, X. Zhang, I. Lupandina, R. Desquiens, and T. Laneyrid, “Metrics for benchmarking and evaluation of dynamic transformer thermal models,” 2023.
- [25] T. Gradnik, “Cigre dtm benchmarking platform,” 2026. [Online]. Available: <https://gitlab.com/cigrea2.60/cigre-dtm-benchmarking-platform>
- [26] A. Pegoraro, “Power transformers – github,” September 08, 2026. [Online]. Available: <https://github.com/AlphaNightLight/power-transformers>
- [27] T. Chai and R. R. Draxler, “Root mean square error (rmse) or mean absolute error (mae)?—arguments against avoiding rmse in the literature,” *Geoscientific model development*, vol. 7, no. 3, pp. 1247–1250, 2014.
- [28] T. O. Hodson, “Root-mean-square error (rmse) or mean absolute error (mae): when to use them or not,” *Geoscientific model development*, vol. 15, no. 14, pp. 5481–5487, 2022.
- [29] A. Pegoraro, “Power transformers – github pages,” September 08, 2026. [Online]. Available: <https://alphanightlight.github.io/power-transformers>
- [30] J. Kukačka, V. Golkov, and D. Cremers, “Regularization for deep learning: A taxonomy,” *arXiv preprint arXiv:1710.10686*, 2017.

- [31] J. C. Butcher, “A history of runge-kutta methods,” *Applied numerical mathematics*, vol. 20, no. 3, pp. 247–260, 1996.
- [32] J. H. Cartwright and O. Piro, “The dynamics of runge–kutta methods,” *International Journal of Bifurcation and Chaos*, vol. 2, no. 03, pp. 427–449, 1992.
- [33] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud, “Neural ordinary differential equations,” in *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, Eds., vol. 31. Curran Associates, Inc., 2018. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2018/file/69386f6bb1dfed68692a24c8686939b9-Paper.pdf
- [34] A. N. Montanari and L. A. Aguirre, “Observability of network systems: A critical review of recent results,” *Journal of Control, Automation and Electrical Systems*, vol. 31, no. 6, pp. 1348–1374, 2020.

Appendix A

Observability in RNNs

The material presented in the appendix is not directly related to the research question, and the preliminary conclusion require additional verification before being considered scientifically relevant. However, the numerous questions concerning the topic received during the thesis' presentation, together with the relevance that observability may play in future works motivates the presence of the extra material presented here. Section A.1 provides a brief introduction to observability, while Section A.2 discusses some preliminary impacts the RNNs can have on it.

A.1 Definition of Observability

A dynamical system is *observable* if it is possible to reconstruct the evolution of the state vector θ from the knowledge of the input vector u and the corresponding measurable output vector θ_o over a certain time [34]. Giving the precise definition of observability is beyond the purpose of the report, but when the system is linear and in SSR then a necessary and sufficient condition can be checked:

A linear system in State Space Representation is observable if and only if its observability matrix is full-rank.

$$O = \begin{bmatrix} C \\ CA \\ CA^2 \\ \dots \\ CA^{n-1} \end{bmatrix} \in \mathbb{R}^{n \times n} \quad (\text{A.1})$$

For a generic system with $A \in \mathbb{R}^{n \times n}$ and $C \in \mathbb{R}^{1 \times n}$ the observability matrix is defined as in Equation A.1, while Equation A.2 reports the specific case for $n = 3$, which is the one concerned by the project.

$$O = \begin{bmatrix} C \\ CA \\ CA^2 \end{bmatrix} \in \mathbb{R}^{3 \times 3} \quad (\text{A.2})$$

A.2 Observability Improvements with RNNs

IEC Model

The numerical values of A and C for the baseline IEC Model and the parameters defined in TTSa are the following:

$$A_{IEC} = \begin{bmatrix} -1.333\text{e-}2 & 0 & 0 \\ 0 & -7.143\text{e-}2 & 0 \\ 0 & 0 & -1.333\text{e-}2 \end{bmatrix}, \quad C = [1 \quad 0 \quad 0]$$

Employing Equation A.2 it is possible to derive the observability matrix corresponding to the analyzed model:

$$O_{IEC} = \begin{bmatrix} 1 & 0 & 0 \\ -1.333\text{e-}2 & 0 & 0 \\ +1.778\text{e-}4 & 0 & 0 \end{bmatrix}$$

The matrix has rank 1 but dimension 3, hence the observability condition is *not* satisfied. It means there exist situations in which the knowledge of u and θ_o is not sufficient to properly reconstruct all the three components of the state vector θ . However, it may still be possible to reconstruct some of them; it is the reason why Experiment I presented in Section 5.1 could determine θ_o with high precision but had only a marginal impact on θ_h .

RNN_p6

Unlike DCT, the RNN architectures cause modifications to the system matrix A , and consequently to the observability of the system. Taking into account the improvement matrix A' suggested by the best RNN, which is RNN_p6, the following system matrix A_{p6} together with the associated observability matrix O_{p6} is obtained:

$$A_{p6} = \begin{bmatrix} -1.083\text{e-}2 & -2.383\text{e-}3 & 0 \\ 0 & -7.143\text{e-}2 & -1.228\text{e-}2 \\ 0 & 0 & -2.238\text{e-}2 \end{bmatrix}$$

$$O_{p6} = \begin{bmatrix} 1 & 0 & 0 \\ -1.083\text{e-}2 & -2.383\text{e-}3 & 0 \\ +1.173\text{e-}4 & +1.961\text{e-}4 & +2.928\text{e-}5 \end{bmatrix}$$

The output matrix C used in the computation of O_{p6} is again $[1 \ 0 \ 0]$, since it is not affected by **RNNs**. The observability matrix is now full-rank, hence the modifications proposed by **RNN_p6** make the system observable. The shift from unobservable to observable can be the motivation of the different performances of the Luenberger Observer: when deployed on the IEC Model, the unobservability allow it to estimate only θ_o , while the deployment on the observable system generated by **RNN_p6** can properly retrieve also the two hotspot components causing better approximations.

RNN_p5

Another interesting behavior is the one presented by **RNN_p5**. The system matrix and the observability matrix generated by its improvement are the following:

$$A_{p5} = \begin{bmatrix} -1.064\text{e-}2 & -1.278\text{e-}3 & 0 \\ 0 & -7.143\text{e-}2 & 0 \\ 0 & -3.721\text{e-}3 & -2.322\text{e-}2 \end{bmatrix}$$

$$O_{p5} = \begin{bmatrix} 1 & 0 & 0 \\ -1.064\text{e-}2 & -1.278\text{e-}3 & 0 \\ +1.132\text{e-}4 & +1.049\text{e-}4 & 0 \end{bmatrix}$$

The rank of O_{p5} is 2, which implies unobservability but is higher than the one of the IEC Model. Since **RNN_p5** has an intermediate performance between **RNN_p6** and the baseline model, it is reasonable to formulate the hypothesis that the rank of the observability matrix plays a role in the model estimation performance. Confirm or reject the hypothesis is out of the scope of the thesis. More in general, analyze the effects observability causes to Power Transformer models can be a valuable future work, as it would be useful to investigate the factors that lead a **RNN** to generate an observable or unobservable system matrix.

